WNDRVR
SUPPORT NETWORK

VXWORKS 7 USB HOST STACK PROGRAMMER'S GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 USB Host Stack Programmer's Guide, SR0650

30 June 2020

1. USB HOST STACK

1. Support for the USB Host Stack

The VxWorks® USB host stack is one of the three primary stacks for USB and is required by all other host packages.

For more information on building packages for USB, see the VxWorks Source Build Options for USB Support.

1. Create the VxWorks Source Build on page 1To add support for the USB host stack to VxWorks, you must first create a VxWorks Source Build (VSB), which allows you to build the appropriate VxWorks kernel libraries.
2. Create the VxWorks Image Project on page 1After creating a VxWorks Source Build (VSB) with support for the the USB host stack, use that VSB to create the VIP project.
3. Setting the USB Initial Memory Partition Size on page 2You can configure the size of a dedicated memory partition, which is used internally by the USB host stack.
4. Configuring Bus Tasks on page 3You can configure the time delay that the bus events monitoring task uses between successive events polling.

1.1. Create the VxWorks Source Build

To add support for the USB host stack to VxWorks, you must first create a VxWorks Source Build (VSB), which allows you to build the appropriate VxWorks kernel libraries.

Procedure

1. Create a VSB project.

   Create a VSB project based on a BSP or a CPU architecture. For general instructions on how to build a VSB project, see the VxWorks 7 Configuration and Build Guide.

2. Include the following required configuration options in your VSB.

   Enable following packages. For more information on each package, see the VxWorks Source Build Options for USB Support.

   - **USB_CORE**
   - **USB_HOST**
   - **USB_HCORE**

   After creating the VSB, create a VxWorks Image Project (VIP).

Parent topic: Support for the USB Host Stack on page 1

Next topic: Create the VxWorks Image Project on page 1

1.2. Create the VxWorks Image Project

After creating a VxWorks Source Build (VSB) with support for the the USB host stack, use that VSB to create the VIP project.

About This Task

A VIP allows you to build the VxWorks run-time kernel image based on the kernel libraries included in your VSB project.

Procedure

1. Create a VIP.

   Create a VIP based on the VSB created in Create the VxWorks Source Build on page 1. For general instructions on how to build a VIP, see the VxWorks 7 Configuration and Build Guide.

2. Include the required components in the VIP.

   Add the following component to your VIP:

   - **INCLUDE_USB**

3. Set the **INCLUDE_USB** configuration parameters.

   For more information on these parameters and their default values, see the *Wind River USB for VxWorks 7 Component Reference*, Setting the USB Initial Memory Partition Size on page 2, and Configuring Bus Tasks on page 3.

4. Include the **INCLUDE_USB_INIT** component.

   This is the initialization component for the USB and provides support for automatic initialization at startup. This component is required by all host controller initialization components. For more information, Initializing the USBD on page 5.

Parent topic: Support for the USB Host Stack on page 1

Previous topic: Create the VxWorks Source Build on page 1
Next topic: Setting the USB Initial Memory Partition Size on page 2

1.3. Setting the USB Initial Memory Partition Size

You can configure the size of a dedicated memory partition, which is used internally by the USB host stack.

About This Task

To change the initial memory partition from the default of 0x200000 bytes (2 Mbytes), set the **INCLUDE_USB** component configuration parameter **USB_MEM_PART_SIZE**. The USB host stack creates this memory partition before initialization of other USB host stack components.

However, be aware that the **USB_MEM_PART_SIZE** configuration parameter value applies only to the initial USB memory partition size. The USB system allows devices to be attached or removed dynamically. Each attached device requires some memory to operate. The memory is released when the device is removed. As more devices are attached to the system, memory used by the USB stack may increase and the available remaining memory can become insufficient. When this occurs, the USB stack increases the dedicated USB memory partition from the system memory partition if the system memory partition is still allocable.

Parent topic: Support for the USB Host Stack on page 1

Previous topic: Create the VxWorks Image Project on page 1
Next topic: Configuring Bus Tasks on page 3

1.4. Configuring Bus Tasks

You can configure the time delay that the bus events monitoring task uses between successive events polling.

About This Task

For each host controller attached to the USBD, the hub class driver spawns a bus task responsible for monitoring bus events, such as the attachment and removal of devices. Each USBD bus task has the VxWorks task name **BusM A**, **BusM B**, and so on. These tasks are normally dormant—that is, consuming no CPU time—and they typically wake up only when a USB hub reports a change on one of its ports.

The bus events monitoring task has a polling interval for detecting bus events. This interval is determined by the **INCLUDE_USB** component configuration parameter **USB_HUB_POLLING_INTERVAL**. The default value is 2, which means a 2 milliseconds task delay between successive events polling.

Parent topic: Support for the USB Host Stack on page 1

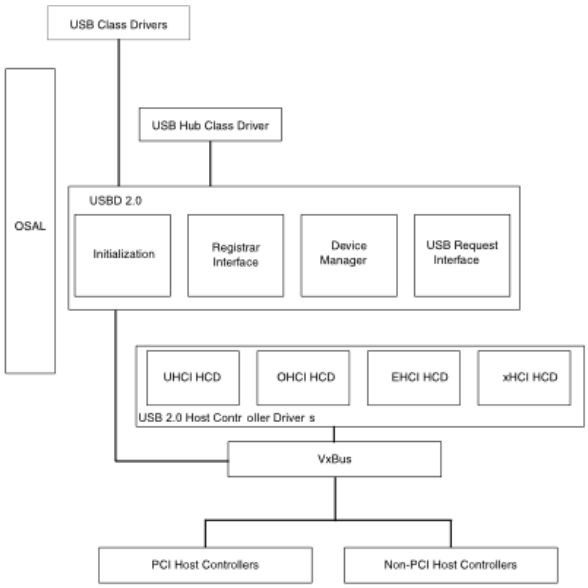
Previous topic: Setting the USB Initial Memory Partition Size on page 2

2. The USBD Interface

The USB host driver (USBD) is an interface layer that acts as a channel between class drivers and host controller drivers.

Figure 1 on page 3 shows these interrelationships of the various parts that comprise the USBD.

Figure 1. USB Host Stack



USBD is responsible for power management, USB bandwidth management, and dynamic device connection and disconnection. It manages each USB peripheral device connected to the host and provides the path through which higher layers communicate with the USB. USBD also handles all standard requests and routes other types of requests to the class drivers. USBD supports the USB 3.0 specification and is backward compatible with USB 2.0 and USB 1.1. The USBD is handled transparently by the USB hub class driver, which controls the functionality of a USB hub. This driver is a class of USB device that provides extension ports for connection

additional USB devices to the USB host. The root hub, which is controlled by the USB hub class driver, is emulated by the host controller driver. Each HCD can control one or more underlying HCs, and the USBD can connect to multiple HCDs simultaneously.

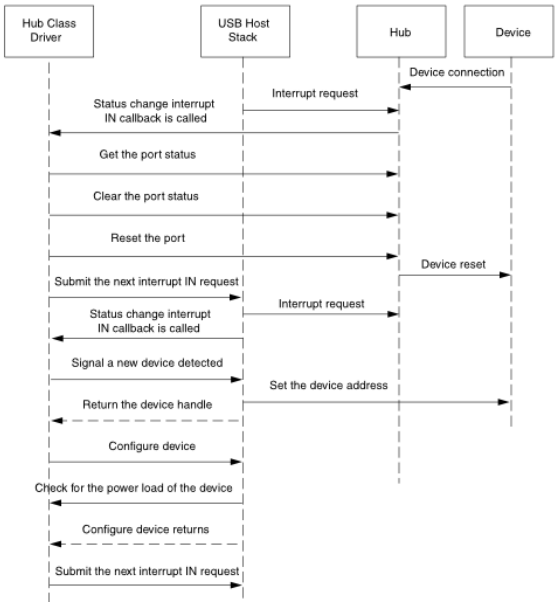
> Note: OSAL provides an abstracted and simplified view of the VxWorks operating system services to the host stack. OSAL includes routines that manage tasks, mutexes, implied semaphores, memory allocation, and system time.

The hub class driver provides an initialization and an exit routine to the USB class drivers and USB applications. It uses these APIs to interact with the USB host stack.

Process for Connecting to and Disconnecting from a USB Port

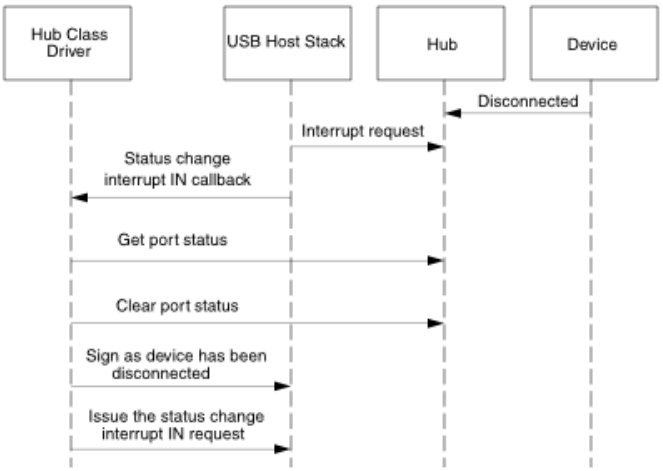
Figure 2 on page 4 shows the connection sequence for a USB device, which involved connecting a USB device to a USB port, connecting the device to the hub, and the USB host stack informing the hub class driver of the connection.

Figure 2. Connecting a Device to a Hub



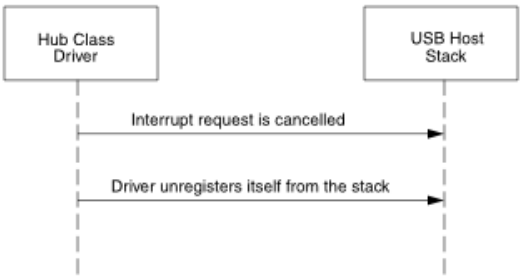
Removing a USB device from a USB port disconnects the device from the hub, and the USB host stack informs the hub class driver, as shown in Figure 3 on page 4.

Figure 3. Removing a Device from a Hub



The hub class driver then calls **usbHubExit()**, which does clean up and calls **usbHstDriverDeregister()** to deregister the it with the USB host stack. The hub class driver then stops polling the root hub's interrupt pipe for changes on the downstream port. Figure 4 on page 5 shows the deregistration process.

Figure 4. Deregistering a Hub Class Driver



1. Initializing the USBD on page 5The USBD is initialized in a multi-step process during the VxWorks boot sequence.
2. Choosing an Initialization Method on page 7There are several acceptable initialization scenarios for USBD.

2.1. Initializing the USBD

The USBD is initialized in a multi-step process during the VxWorks boot sequence.

About This Task

If you include the **INCLUDE_USB_INIT** component in your VIP, the system calls the following sequence of routines, thereby handling initialization automatically. For debugging purposes, you can call the initialization routines manually. You can also call **usbdInit()** more than once, as explained below.

Procedure

1. Initialize internal data structures.

   First, **usbInit()** is called to initialize global data structures, memory, the message library, the hub, the host controller driver, and the USB target. **usbInit()** is defined in *installDir* **/pkgs_v2/connectivity/usb/core/cfg/usrUsbInit.c**.

2. Initialize the USBD.

Next, **usbInit()** calls **usbdInit()**, the USBD entry point, which initializes the USBD. **usbdInit()**, must be called once, to initialize internal USBD data structures. If you want to remove and re-initialize only the USBD (at a later time), you can call **usbdInit()** directly. Thus, in a given system, it is acceptable to call **usbdInit()** once (for example, during the boot sequence) or many times (as during the initialization of each USBD client).

3. Initialize the hub class driver.

   Then, **usbInit()** calls **usrUsbHubInit()**, which calls **usbHubInit()** to initialize the hub class driver global variables and function pointers.

4. Initialize host controllers with USBD.

   usbInit() then calls **usbHcdInit()**, if the individual host controller drivers also included their respective initialization components, such as **INCLUDE_EHCI_INIT** and so on. This routine initializes the host controllers—performs initialization of data structures only —when the system calls the initialization entry point in **usrRoot()** (during the bootup sequence).

   The host controller drivers must be initialized after the USBD.

5. Register the host controllers.
   1. For all host controllers, **usbHcdInit()** calls host controller driver initialization routines.

      For all host controllers, **usbHcdInit()** calls host controller driver initialization routines. Examples of initialization routines are **usbEhcdInit()**, which is the entrypoint for the EHCI initialization function. In the HCD initialize function, **usbHstHCDRegister()** is invoked to register the HCI driver with the USBD.

      2. Register the host controllers with VxBus.

      If the individual host controller drivers included their respective initialization components, such as **INCLUDE_EHCI_INIT** and so on, then **vxbUsbHciRegister()** is called (automatically) from either the target initialization code (bootup) to register the host controller driver with VxBus. If the initialization was not included, then it can be call manually during runtime. This routine internally calls different register functions for different controllers to register with VxBus; for example, EHCI uses **vxbUsbEhciRegister()**.

   VxBus will invoke the internal initialization.

   When the host controllers drivers are registered with VxBus, VxBus in turn announces them to the system and starts them.

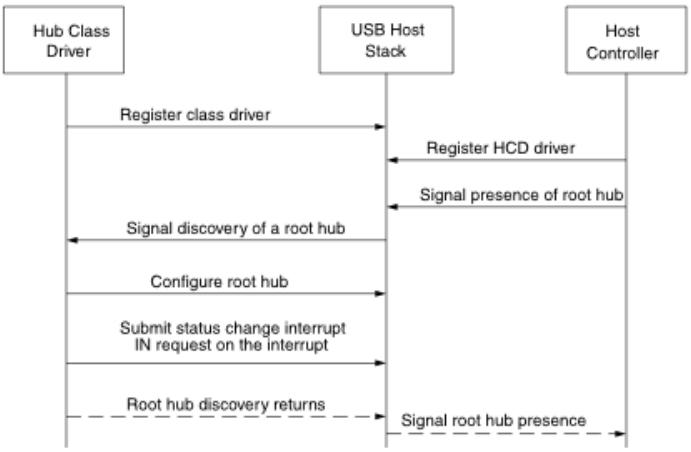
   The driver registers itself with VxBus for both PCI and local bus types, causing VxBus to discover the controller device and execute the appropriate VxBus initialization routines.

6. Register the USB hub class bus type with VxBus.

   After USBD initialization, all the class driver should call usbHstDriverRegister( ) to register itself with USBD, and also to register the class driver with VxBus. See also USB Host Class Driver Registration on page 23.

   Figure 1 on page 6 shows the registration process. After registering itself with the host stack, the hub class driver configures the root hub for all host controller drivers registered with the USBD.

   Figure 5. Registering the Hub Class Driver



Once registered, a device manager provides interfaces to the USBD for the hub class driver for adding, removing, suspending, and resuming a device. For more information, see The USBD Interface on page 3.

Then, the hub class driver starts polling the interrupt pipe of the root hub to monitor changes on the downstream ports.

Parent topic: The USBD Interface on page 3

Next topic: Choosing an Initialization Method on page 7

2.2. Choosing an Initialization Method

There are several acceptable initialization scenarios for USBD.

About This Task

The **usbdInit()** routine must be called before any other controller driver and class or function driver routines, including **usbHubInit()**. However, not all USBD clients must call **usbdInit()** before one or more HCDs have been attached to the USBD. Either of the following initialization sequence scenarios is acceptable.

Note: Calls to **usbdInit()**, **usbHubInit()**, and **usbHcdInit()** occur before the second level initialization of VxbBus takes place.

Traditional Boot-Time Initialization Scenario

In this scenario, each host controller driver calls **vxbUsb***Controller***Register()** to register with VxBus. Then the calls are made to **usbdInit()** and **usbHubInit()**. The desired host controller driver is initialized and registered with Uthe SBD. VxBus detects the host controllers and calls each Probe and Connect routine. The host controller devices are up and ready for attaching to USB devices. Finally, the USB class driver initialization entry point is called.

Hot Swap-Driven Initialization Scenario

In this scenario, once the VxWorks image project (VIP) is created with proper support for the host stack and host controllers, the hot swap code calls **vxbDevInit()** which calls **usbInit()**. Then, the hot swap code calls **usb***Hcd***Init()**, registers the host controller driver with USBD. Then, the hot swap code calls **vxbUsb***Controller***Register()**, which registers the host controller drivers with VxBus. Finally, if VxBus detects the appropriate hardware, it starts that host controller driver.

Parent topic: The USBD Interface on page 3

Previous topic: Initializing the USBD on page 5

2. USB HOST CONTROLLERS

1. USB Host Controllers Overview

USB host controllers are the hardware components responsible for controlling USB on the board. USB-enabled hardware systems include one or more USB host controllers.

The desired host controller is added to the VIP, once the VSB is created with the proper support. For details on which components are required for each specific host controller, refer to the documentation for the individual host controller package.

More than one host controller can be present in the image at once. In fact, some EHCI systems may require that additional host controllers be included. For more information, see ECHI Configuration Tips on page 17.

Prebuilt Host Controllers Drivers

Most manufacturers produce USB host controllers that conform to one or more of the major device specifications, including the following:

- the Enhanced Host Controller Interface (EHCI)
- the Open Host Controller Interface (OHCI)
- the Universal Host Controller Interface (UHCI)
- the eXtensible Host Controller Interface (xHCI)
- the Synopsys Host Controller Interface (SYNOPSYSHCI)

For each type of host controller there is a single, hardware-dependent USB host controller driver (HCD). Wind River provides the source for the EHCI prebuilt drivers in the following directories:

- *installDir* **/pkgs_v2/connectivity/usb/ctlr/ehci** for EHCI
- *installDir* **/pkgs_v2/connectivity/usb/ctlr/ohci** for OHCI
- *installDir* **/pkgs_v2/connectivity/usb/ctlr/uhci** for UHCI
- *installDir* **/pkgs_v2/connectivity/usb/ctlr/xhci** for xHCI
- *installDir* **/pkgs_v2/connectivity/usb/ctlr/mhdrc** for Mhdrc
- *installDir* **/pkgs_v2/connectivity/usb/ctlr/dwc2dr** for SynopsysHCI

Defined Limits for Host Controllers

The default limit of detectable USB host controllers is hardcoded in the *installDir* **/pkgs_v2/connectivity/usb/core/h/usbOsal.h** header file. For most host controllers, the maximum number is 20.

If your hardware has more host controllers than the default limits, you must edit **usbOsal.h** and rebuild the VSB.

Including Multiple Bus Instances on page 9 For USB 3.0, the Wind River USB 3.0 stack supports a USB Host Controller instance that can have multiple bus instances with different speeds.

1.1. Including Multiple Bus Instances

For USB 3.0, the Wind River USB 3.0 stack supports a USB Host Controller instance that can have multiple bus instances with different speeds.

About This Task

In the current version of Wind River USB, there is a 1-N mapping between the USB Host Controller instance and USB bus instances. In previous versions of the Wind River USB host stack, there was a 1-1 mapping between the entry in **gUSBBusInfoList[]** and the Host Controller Driver private data structure representing the Host Controller instance (the **USB_EHCD_DATA** data structure instance).

Adding or Removing Host Controller and Bus Instances Separately

As part of the USB 3.0 Dual Bus Architecture support, the following APIs have been added to allow adding or removing host controller instances and bus instances separately:

- **usbHstCtlrAdd()**
- **usbHstCtlrRemove()**
- **usbHstBusAdd()**
- **usbHstBusRemove()**

In most cases, driver designers can continue to use **usbHstBusRegister()** and **usbHstBusDeregister()** with the new APIs because they are, in fact, wrappers around the new APIs. However, if a host controller instance contains several bus instances, the driver should use the new APIs in the following fashion:

Procedure

1. Call **usbHstCtlrAdd()** to add a new HC instance.
2. Call **usbHstBusAdd()** to add a new bus instance for each bus instance.

   You call **usbHstBusAdd()** with different bus speeds as the distinguishing factor. This means that **usbHstBusAdd()** can be called multiple times.

3. To remove a host controller instance, use the API counterpart to adding it.
   1. Call **usbHstBusRemove()** for each bus instance.

      When calling **usbHstBusRemove()**, use different bus speeds to identify each instance.

      2. Call **usbHstCtlrRemove()** to remove the host controller instance.

Parent topic: USB Host Controllers Overview on page 9

2. Writing a Custom Host Controller
2.1. Implement Routines for the HCD Structures

To add a new HCD for a host controller that Wind River does not already support, the first step is to implement routines for the HCD structure.

Procedure

1. Implement routines, as needed.

The USBD layer communicates with the HCD via routines assigned to the function pointers contained in the **USBHST_HC_DRIVER** structure. This structure is defined in the **usbHst.h** and is populated and registered with the USB host stack during HCD initialization.

Modify the routines in this structure according to the routine descriptions, and as needed for your environment.

getFrameNumber()

This routine obtains the current frame number from the frame number register.

setBitRate()

If this functionality is supported, this routine modifies the frame width, which can be incremented or decremented by one bit time, once in every six frames.

isBandwidthAvailable()

This routine determines if there is enough bandwidth to support the new configuration, or an alternate interface setting. It parses the configuration or interface descriptor. When the configuration descriptor is passed as a parameter, it includes all interface and endpoint descriptors. Similarly, when the interface descriptor is passed as a parameter, it includes all endpoint descriptors corresponding to that interface.

createPipe()

This routine parses and creates a pipe for the specified USB endpoint descriptor. The pipe is then added to a queue of pipes maintained for transfers.

modifyDefaultPipe()

This routine modifies the properties of the default pipe (address 0, endpoint 0).

deletePipe()

This routine deletes a pipe corresponding to an endpoint. It searches the list of pipes for the pipe corresponding to the given endpoint, deletes it, and updates the list. All pending transfers for the pipe are cancelled and the callback routines for the transfers are called.

pipeControl()

This routine controls the pipe characteristics using the **USBHST_PIPE_CONTROL_INFO** parameter. For details, see Support for New HCD Pipe Control interface.

isRequestPending()

This routine checks for requests pending for an endpoint and returns a value accordingly. If there are pending requests, it returns a **Success** status.

submitURB()

This routine submits a request to the endpoint. It populates any host controller-specific data structures to submit a transfer. The new request is added to the queue of requests pending for the endpoint.This routine is implemented as a non-blocking routine. When a request completes, the callback routine for the request is called.

cancelURB()

This routine cancels a request submitted to an endpoint. It searches the queue of requests pending for a given endpoint. If the desired request is pending for the endpoint, the request is cancelled and the queue of requests for the endpoint is updated.

clearTTRequestComplete() and resetTTRequestComplete()

These routines handle errors that occur during split transactions. They are implemented only for an EHCI host controller.

2. For an EHCD driver, implement additional routines.

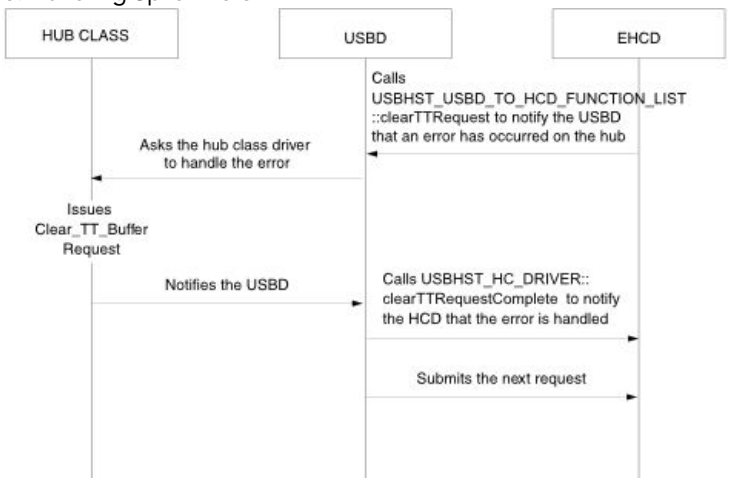
WNDRVR

The EHCD maintains an additional interface, the **USBHST_USBD_TO_HCD_FUNCTION_LIST** structure, defined in *installDir* **/target/h/usb/usbHst.h**. This structure contains function pointers to two routines, listed below.

- **clearTTRequest()**, called by the HCD to submit a clear TT request
- **resetTTRequest()**, called by the HCD to submit a reset TT request

This data structure is registered with the HCD by the USBD. If an error occurs on the hub, the EHCD does not handle any request for that hub, unless the error is handled, that is, unless the transaction translator (TT) buffers are cleared. Figure 1 on page 12 is a state diagram that explains how the split errors are handled using above interface. The **Reset_TT** request is handled similarly.

Figure 6. Handling Split Errors



3. Use HCD error codes to report HCD errors when implementing routines.

   According to Wind River convention, the HCD sets the system errno whenever it returns an error. HCDs return the standard VxWorks constant **USBHST_SUCCESS** when a routine completes successfully. HCD error codes are defined in **usbHst.h** as shown below.

```
/************** Different USB status codes - Start ****************************/
#define USBHST_STATUS                        INT8
#define USBHST_SUCCESS                       ((INT8)0)
#define USBHST_TRANSFER_COMPLETED            ((INT8)1)
#define USBHST_FAILURE                       ((INT8)-1)
#define USBHST_MEMORY_NOT_ALLOCATED          ((INT8)-2)
#define USBHST_INSUFFICIENT_BANDWIDTH        ((INT8)-3)
#define USBHST_INSUFFICIENT_MEMORY           ((INT8)-4)
#define USBHST_INSUFFICIENT_RESOURCE         ((INT8)-5)
#define USBHST_INVALID_REQUEST               ((INT8)-6)
#define USBHST_INVALID_PARAMETER             ((INT8)-7)
#define USBHST_STALL_ERROR                   ((INT8)-8)
#define USBHST_DEVICE_NOT_RESPONDING_ERROR   ((INT8)-9)
#define USBHST_DATA_OVERRUN_ERROR            ((INT8)-10)
#define USBHST_DATA_UNDERRUN_ERROR           ((INT8)-11)
#define USBHST_BUFFER_OVERRUN_ERROR          ((INT8)-12)
#define USBHST_BUFFER_UNDERRUN_ERROR         ((INT8)-13)
#define USBHST_TRANSFER_CANCELLED            ((INT8)-14)
#define USBHST_TIMEOUT                       ((INT8)-15)
#define USBHST_BAD_START_OF_FRAME            ((INT8)-16)
```

2.2. Register Host Controller Instances and Buses

For new host controllers written by customers, you need to handle the registration, which is otherwise done automatically by the configuration infrastructure.

About This Task

Note: It is sometimes preferable to call routines manually, especially for debugging purposes.

Procedure

1. Register the host controller driver with USBD.

   All HCDs are required to register with the USBD by calling the routine **usbHstHCDRegister()**, which is documented in the online USB API Reference.

   ```
   status = usbHstHCDRegister(&g_pEHCDriverInfo,
                              &g_EHCDHandle,
                              &g_USBDRequestFunctions,
                              VXB_BUSID_USB_HOST_EHCI
                              );
   ```

   This routine takes a pointer to a host controller driver—the **USBHST_HC_DRIVER** structure—as the first argument to this routine. The **USBHST_USBD_TO_HCD_FUNCTION_LIST** structure is passed to **&g_USBDRequestFunctions** as the third parameter. For an example, see **usbEhcdInitExit.c** in *installDir* **/vxworks-7/pkgs_v2/connectivity/usb/ctlr/ehci/src**.

2. Register a bus for the host controller driver with the USBD.

   All HCDs are required to register a bus, with the USB host stack, for the host controller, by calling **usbHstBusRegister()**. For example, assuming that there are three EHCI host controllers on the system, the following actions occur.

   1. **usbHstHCDRegister()** is called once for each EHCI host controller, to register the host controller itself.
   2. **usbHstBusRegister()** is called once for each EHCI host controller, to register an USB Bus corresponding to the host controller.

3. Register the host controller with VxBus.

   Call **vxbUsbHciRegister()**, which calls the appropriate register routines for each host controller, to register with VxBus. See Initializing the USBD on page 5 for details.

2.3. Handle Transfer Requests

HCDs are required to emulate the behavior of the root hub by intercepting transfer requests intended for the root hub and synthesizing standard USB responses to these requests.

About This Task

Following is an example of how an HCD would emulate a root hub.

WNDRVR

Procedure

1. Disable ports upon initialization.

   When a host controller is first initialized, the root hub must respond at the default USB address 0 and disable its downstream ports.

2. Respond to configuration requests from the USBD.
   1. The USBD interrogates the root hub, just as it would other hubs, by issuing **USB GET_DESCRIPTOR** requests.
   2. If the hub class driver is registered, the USBD calls it to configure the root hub by issuing a series of **SET_ADDRESS, SET_CONFIGURATION**, and **SET_FEATURE** requests.
   3. The HCD recognizes which of these requests are intended for the root hub, and responds to them appropriately.
3. Intercept and reply to URBs directed to the root hub's interrupt endpoint.
   1. After configuration, the hub class driver begins polling the root hub's interrupt status pipe to monitor changes on the root hub's downstream ports.
   2. The HCD intercepts URBs directed to the root hub's interrupt endpoint and synthesizes the appropriate replies. Typically, the HCD queues requests directed to the root hub separately from those that actually result in bus operations.

2.4. Deregister Host Controller Drivers and Buses

When no longer required, deregister the host controller drivers and buses, as outlined in the following steps.

Procedure

1. Deregister the host controller.

   All HCDs deregister the bus for the host controller, from the USB host stack, by calling usbHstBusDeregister( ). This routine is called for the number of the bus registered. If there are functional devices (active devices or configured devices) on the bus, the bus is not deregistered.

2. Deregister the driver host controller.

   All HCDs deregister themselves from the USB host stack by calling **usbHstHCDDeregister()**. If there are buses registered for the HCD, the HCD is not deregistered.

3. Adding EHCI Host Controller Support

To add support for the EHCI host controller, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the EHCI host controller interface, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP). For more information about configuring host controller interfaces, see the Support for the USB Host Stack on page 1.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the EHCI to the VSB.

   Include the following VSB configuration options:

   - **USB_CCORE**
   - **USB_CTLR**
   - **USB_EHCI**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Include **INCLUDE_EHCI** to the VIP.
5. Include the **INCLUDE_EHCI_INIT** component.

   This component adds initialization for the host controller at startup. For more information, see Initializing the USBD on page 5.

Setting EHCI Controller Properties on page 15 The properties used for EHCI controllers apply only to VxBus with FDT support.
ECHI Configuration Tips on page 17 Because EHCI requires companion host controllers for FULL/LOW speed operation, the configuration methods differ depending upon which controllers and chips you have.

3.1. Setting EHCI Controller Properties

The properties used for EHCI controllers apply only to VxBus with FDT support.

About This Task

To set Freescale SOC USB controller properties, follow the steps below. For more information on using the device tree, see http://www.devicetree.org/ or ePAPR.

Procedure

1. Set the **compatible** property.

   This property specifies the name of the system and is a required property. Options are listed in the table below.

Table 1. Options for EHCI compatible Property

Option	Description
“ehci”	Use for a normal EHCI controller following the EHCI specification.
“fsl-usb2-dr”	Use for freescale dual role USB controllers using as host mode.

Option	Description
“fsl-usb2-mph”	Use for freescale multi-port host USB controllers.
“fsl,ehci”	Use for freescale multi-port host USB controllers or freescale dual role USB controllers using as host mode.

2. Set the **reg** property.

   This property specifies the offset and length of the register set for the device and is a required property.

3. Set the **interrupts** property.

   This is a required property and is set using the format **<***a b***>** or **<***a b c d***>**, where *a* is the interrupt number and *b* (or *b c d*) is a field representing an encoding of the sense and level information for the interrupt. This should be encoded depending on the type of interrupt controller you have.

4. Set the **interrupt-parent** property.

   This property is required and specifies the **phandle** for the interrupt controller that services interrupts for this device.

5. Set the **usb-phy** property.

   It is recommended that you set this property, which specifies the **phandle** of the USB PHY controller. If this property not defined, then the USB PHY controller is automatically usable without any specific configuration, and the **phy_type** property is set to “ulpi” by default.

   Note: For the Freescale EHCI controller, use “fsl, usbphy” as the property.

6. Optionally, set the **clocks** property.

   This property specifies the list of clock source phandle and clock parameter used by the USB PHY controller. This property is optional, and you can ignore it if you do not need it.

   For more information, see the **vxbClkLib** entry in the online *Wind River USB API Reference*.

7. Optionally, set the **pinmux** property.

   This property specifies the pin mux phandle used by the USB PHY controller. This property is optional, and you can ignore it if you do not need it. The format for this property is:

```
pinmux-n[0, 1, 2, …]
```

   For more information, see the **vxbPinMuxLib** entry in the online *Wind River USB API Reference*.

UTMI Freescale On-Chip PHY USB Controller Device Node

This example is for a multi-port host USB controller device node, which uses a UTMI freescale on-chip PHY:

```
        usbphy2: usbphy@214100 {
                compatible = "fsl,on-chip-phy";
                reg = <0x214100 0x100>;
                phy_type = "utmi";
```

```
                };
        usb@211000 {
                #address-cells = <1>;
                #size-cells = <0>;
                reg = <0x211000 0x1000>;
                compatible = "fsl-usb2-mph";
                interrupts = <45 2>;
                interrupt-parent = <&mpic>;
                fsl,usbphy = <&usbphy2>;
                };
```

ULPI Type USB Controller Device Node

This example is for a dual role USB controller device node, which is ULPI type, and is ready to use without any specific configuration after powering on.

```
        usb@23000 {
        compatible = "fsl-usb2-dr";
                reg = <23000 1000>;
                #address-cells = <1>;
                #size-cells = <0>;
                interrupt-parent = <700>;
                interrupts = <26 1>;
        };
```

Parent topic: Adding EHCI Host Controller Support on page 14

Related information
ECHI Configuration Tips on page 17

3.2. ECHI Configuration Tips

Because EHCI requires companion host controllers for FULL/LOW speed operation, the configuration methods differ depending upon which controllers and chips you have.

FULL/LOW speed host controllers (such as OHCI/UHCI), since they come into play earlier than EHCI, do not require EHCI, however, ECHI does require companion host controllers for FULL/LOW speed operation.

If a system has EHCI, along with its companion chips (typically UHCI in a PC), then the situation is complicated by the "Port Ownership" switch mechanism. The EHCI has a register **CONFIGFLAG** which has a **CF** bit. Once set, the EHCI takes over the port ownership once any device is attached, so the device first appears to the EHCI controller, the EHCI controller does some speed detection, and if it is found that the device is FULL/LOW speed, the EHCI (actually the EHCI driver, which in VxWorks is the EHCD component) hands over the port ownership to the companion chips (by modifying the **PORTSC** register bits).

Consider a typical PC system. The BIOS has a feature called "boot from USB," and could also have a mini USB stack which has enabled both EHCI and its companion chips. In the BIOS, the EHCI **CONFIGFLAG CF** bit is set. Be sure that BIOS settings agree with the VxWorks configuration settings. For example, if you set the EHCI **CONFIGFLAG CF** bit in the BIOS, but do not include the EHCI component in VxWorks, the hardware reports device connections to EHCI but no driver exists to handle them.

Summary of Configuration Options

The configuration you set depends upon which controllers and chips you have.

Only UHCI/OHCI controllers

For a system with only UHCI/OHCI controllers, include only the UHCD/OHCD components.

An EHCI controller and no companion chip

For a system with only an EHCI controller and no companion chip—such as the Freescale boards with EHCI variation and Embedded TT—only EHCD is required.

EHCI and companion chips and BIOS code

For a system with both EHCI and companion chips, and BIOS code (or pre-VxWorks boot loaders) that has already initialized the EHCI controller, you have two options if you want to use only the companion chips for FULL/LOW speed operations in VxWorks:

- Include EHCD as well, even though all devices are FULL/LOW speeds, the EHCD does some initial port ownership switching before the UHCD/OHCD can detect these FULL/LOW speed devices.
- If you do not want to include EHCD since it may add some more memory, you have to add some code (better in the bootrom, before the USB stack is used) to disable the already initialized EHCI by clearing the CF bit.

EHCI and companion chips, but no BIOS code or boot loaders

For a system with both EHCI and its companion chips, and nos BIOS code or pre-VxWorks boot loaders to initialize the EHCI, then the EHCI companion chips can be used independently with EHCI.

Parent topic: Adding EHCI Host Controller Support on page 14

Related information
Setting EHCI Controller Properties on page 15

4. Adding OHCI Host Controller Support

To add support for the OHCI host controller, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the OHCI host controller interface, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP). For more information about configuring host controller interfaces, see the Support for the USB Host Stack on page 1.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the OHCI to the VSB.

   Include the following VSB configuration options:

   - **USB_CCORE**

- **USB_CTLR**
- **USB_OHCI**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Include **INCLUDE_OHCI** to the VIP.
5. Include the **INCLUDE_OHCI_INIT** component.

   This component supports initialization for the host controller at startup. For more information, see Initializing the USBD on page 5.

5. Adding UHCI Host Controller Support

To add support for the UHCI host controller, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the UHCI host controller interface, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP). For more information about configuring host controller interfaces, see the Support for the USB Host Stack on page 1.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the UHCI to the VSB.

   Include the following VSB configuration options:

   - **USB_CCORE**
   - **USB_CTLR**
   - **USB_UHCI**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Include **INCLUDE_UHCI** to the VIP.
5. Include the **INCLUDE_UHCI_INIT** component.

   This component supports initialization for the host controller at startup. For more information, see Initializing the USBD on page 5.

6. Adding xHCI Host Controller Support

To configure a VxWorks image project with support for the USB eXtensible host controller interface (xHCI), include the appropriate components.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the xHCI host controller interface, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP). For more information about configuring host controller interfaces, see Support for the USB Host Stack on page 1.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the xHCI to the VSB.

   Include the following VSB configuration options:

   - **USB_CCORE**
   - **USB_CTLR**
   - **USB_XHCI**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Include **INCLUDE_USB_XHCI_HCD** to the VIP.
5. Include the **INCLUDE_USB_XHCI_HCD_INIT** component.

   This component supports initialization for the host controller at startup. For more information, see Initializing the USBD on page 5.

7. Adding Mentor Graphics Host Controller Support

To configure a VxWorks image project with support for the Mentor Graphics host controller interface or target role driver, include the appropriate components and set values for parameters.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the Mentor Graphics host controller interface, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP). For more information about configuring host controller interfaces, see Support for the USB Host Stack on page 1.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for USB Mentor Graphics to the VSB.

   Include the following VSB configuration options,

   - **USB_CCORE**
   - **USB_CTLR**
   - **USB_MHDRC**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Include **INCLUDE_USB_MHDRC**.
5. Include the **INCLUDE_USB_MHDRC_HCD_INIT** component.

   This component adds initialization for the host controller at startup. For more information, see Initializing the USBD on page 5.

6. Optionally, include **INCLUDE_USB_MHDRC**.

   Include this component to use USB MUSB HCD, TCD, or OCD.

7. Optionally, include **INCLUDE_MHDRC_CPPI_DMA**.

   Include this component for CPPI DMA mode support.

8. Optionally, include **INCLUDE_USB_PHY_AM335X**.

   Include this component for MUSB on an am335x target.

9. Optionally, include support for the target role driver.

   To include support for the Mentor Graphics Multi-point High-Speed Dual Role USB Controller (MUSB MHDRC) target role driver, include the **INCLUDE_USB_MHDRC_TCD_INIT** component in your VIP.

8. Adding Synopsys Host Controller Support

To configure a VxWorks image project with support for the Synopsys host controller interface or target role driver, include the appropriate components and set values for parameters.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the Synopsys host controller interface, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP). For more information about configuring host controller interfaces, see Support for the USB Host Stack on page 1.

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the xHCI to the VSB.

   Include the following VSB configuration options:

   - **USB_CCORE**
   - **USB_CTLR**
   - **USB_DWC2DR**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Include **INCLUDE_SYNOPSYSHCI**.
5. Include the **INCLUDE_SYNOPSYSHCI_INIT** component.

   This component supports initialization for the host controller at startup. For more information, see Initializing the USBD on page 5.

3. USB HOST CLASS DRIVERS

1. USB Host Class Drivers Overview

USB class drivers are responsible for managing types of devices that can be connected to the USB.

The class driver modules are implemented under the following directory:

installDir **/pkgs_v2/connectivity/usb/host/class**

The drivers rely on the USBD to provide the communication path to each device. For more information, see The USBD Interface on page 3.

USB Host Class Driver Registration on page 23 In order to communicate with USB devices, client modules must register the USBD by calling **usbHstDriverRegister()**. All new class drivers must register with the USBD.
Setting the Device Type on page 24 As part of populating the **USBHST_DEVICE_DRIVER** device driver data structure is to specify the device type.
Defining Event Callback Routines on page 26 As part of populating the **USBHST_DEVICE_DRIVER** device driver data structure, you must define event callback routines.
Registering Vendor-Specific Devices on page 27 In some cases, a single driver can support devices that have similar functionality but different vendor IDs and product IDs.
Using Standard Request Interfaces on page 27 The USB specification defines a set of standard requests, a subset of which must be supported by all USB devices.
Using Data Transfer Interfaces on page 29 Once a client has configured a device, the client can begin to exchange data with that device using the data transfer interfaces provided by the USBD.

1.1. USB Host Class Driver Registration

In order to communicate with USB devices, client modules must register the USBD by calling **usbHstDriverRegister()**. All new class drivers must register with the USBD.

Once registered, the USBD then allocates per-client data structures that are later used to track all requests made by that client. Thus, when a new device is attached, the USBD can associate it with the proper class driver. When a client no longer intends to use the USBD, it calls **usbHstDriverDeregister()** to release its per-client data.

The **usbHstDriverRegister()** routine is further documented in the USB API reference and takes three arguments, described below.

- The first argument is a pointer to the device driver data structure, **USBHST_DEVICE_DRIVER**.
- The second argument is used only for the USB hub class driver, and is handled internally, so is not applicable for normal USB class drivers.
- The third argument is the name of the class driver, which is only relevant for the purpose of showing information.

  For an example using a mouse class driver, see the section on creating a skeleton initialization routine..

The **USBHST_DEVICE_DRIVER** data structure is defined in *installDir* **/pkgs_v2/connectivity/usb/core/h/usbHst.h** as follows:

```
typedef struct usbhst_device_driver
    {
    struct  vxbDevRegInfo vxbDriverInfo;  /* to hold DRIVER_REGISTRATION */
                                          /* information */
    BOOL    bFlagVendorSpecific;  /* Vendor Specific or class specific flag  */
    UINT16  uVendorIDorClass;     /* Vendor ID (if vendor) or Class Code     */
    UINT16  uProductIDorSubClass; /* Dev ID (if vendor) or SubClass Code     */
```

```
    UINT16  uBCDUSBorProtocol;    /* DevRel num (if vendor) or Protocol code */

    VOID *  pDeviceList;          /* Pointer to list of supported devices */
    /*
     * Function registered as to be called when a matching interface/device
     * is connected
     */
    USBHST_STATUS  (*addDevice)   (UINT32    hDevice,
                                   UINT8     uInterfaceNumber,
                                   UINT8     uSpeed,
                                   void      **pDriverData);
    /*
     * Function registered as to be called when a matching interface/device
     * is disconnected
     */
    void          (*removeDevice)  (UINT32    hDevice,
                                    void *    pDriverData);
    /*
     * Function registered as to be called when a matching interface/device
     *  is suspended
     */
    void          (*suspendDevice) (UINT32    hDevice,
                                    void *     pDriverData);

    /*
     * Function registered as to be called when a matching interface/device
     * is resumed
         */
    void          (*resumeDevice)  (UINT32    hDevice,
                                    void *     pDriverData);
    } USBHST_DEVICE_DRIVER, *pUSBHST_DEVICE_DRIVER;
```

This structure must be populated by the client to identify the devices that the driver manages (Setting the Device Type on page 24) and define the callback routines that are invoked at certain USB events (Defining Event Callback Routines on page 26). If a single driver supports devices that have similar functionality, but different vendor IDs and product IDs, you may need to register a list of these devices (Registering Vendor-Specific Devices on page 27).

Parent topic: USB Host Class Drivers Overview on page 23

Related information

Setting the Device Type on page 24
Defining Event Callback Routines on page 26
Registering Vendor-Specific Devices on page 27
Using Standard Request Interfaces on page 27
Using Data Transfer Interfaces on page 29

1.2. Setting the Device Type

As part of populating the **USBHST_DEVICE_DRIVER** device driver data structure is to specify the device type.

About This Task

The Wind River USB host stack provides support to register two types of USB devices with the USB subsystem. These two device types are:

- Vendor-specific devices, which do not adhere to the standard USB class specification and their implementations are vendor-specific. Vendor-specific devices are identified by vendor ID, device ID, and binary-coded decimal (BCD) protocol.
- Class-specific devices, which adhere to the standard USB class specification supported by http://www.usb.org/. Class-specific devices are identified by class, subclass, and protocol.

Each of these data structure members must be assigned the proper value prior to registering the driver.

Procedure

1. Set a value for the **bFlagVendorSpecific** data structure member.

   For vendor specific device drivers, set this flag to TRUE.

   For standard class drivers, set this flag to FALSE .

   According to the USB specification, vendor specific devices set the "Device Class" in the device descriptor as 0xFF. Since the mouse descriptor does not specify 0xFF in the "Device Class" field of the device descriptor, it needs to be FALSE (as a standard class driver).

2. Set a value for the **uVendorIDorClass** data structure member.

   For a vendor specific driver, set this member to the vendor ID.

   For a standard class driver, set this member to the **bInterfaceClass** value in the interface descriptor.

   Remember that mouse devices are HID devices with a class value of 0x03 (defined as **USB2_HID_CLASS** in the Wind River USB stack). Note that this matches the "Interface Class" from the interface descriptor.

3. **uProductIDorSubClass** data structure member.

   For a vendor specific driver, set this member to the product ID code.

   For a standard class driver, set this member to the **bInterfaceSubClass** value in the interface descriptor.

   According to the USB HID 1.11 specification, a value of 0x01 indicates a "Boot Interface subclass" device and matches the "Interface Subclass" in the interface descriptor.

4. Set a value for the **uBCDUSBorProtocol** data structure member.

   For a vendor specific driver, set this member to the **bcdDevice** code, which identifies the device release number for the vendor device.

   For a standard class driver, set this member to the **bInterfaceProtocol** value in the interface descriptor.

   According to the USB HID 1.11 specification, a value of 0x02 (defined as **USB2_MSE_PROTOCOL** in the Wind River USB stack) indicates this is a "mouse" device. It also matches the "Interface Protocol" in the interface descriptor.

5. Set a value for the **pDeviceList** data structure member.

   This is a pointer to an array of type **USBHST_DEVICE_ID**, which contains a list of vendor device IDs and product IDs, specifying the list of devices the driver supports. Using the **uVendorIDorClass**, **uProductIDorSubClass**, **uBCDUSBorProtocol** it can direct the driver to support one vendor device at a time, but if several different vendor devices needed to be registered, this can be a waste of code size and be hard to extend.

   Any of the **uVendorIDorClass**, **uProductIDorSubClass**, and **uBCDUSBorProtocol** can be set to **USBD_NOTIFY_ALL**, which acts as a wildcard for that match item.

Parent topic: USB Host Class Drivers Overview on page 23

Related information
USB Host Class Driver Registration on page 23
Defining Event Callback Routines on page 26
Registering Vendor-Specific Devices on page 27
Using Standard Request Interfaces on page 27
Using Data Transfer Interfaces on page 29

1.3. Defining Event Callback Routines

As part of populating the **USBHST_DEVICE_DRIVER** device driver data structure, you must define event callback routines.

About This Task

These callback routines notify the USBD client whenever a device of a particular type is attached or removed. The **USBHST_DEVICE_DRIVER** data structure includes a data structure member for each of these callbacks, and each must be assigned the proper value prior to registering the driver.

For examples, see the mouse class driver example document.

Procedure

1. Define the *addDevice* callback.

   This callback is invoked each time a matching USB device is inserted. This is the first time the class driver sees the device instance. Class drivers should examine the device characteristics to see if this device has the expected features to work as the device class that is supported by the class driver.

2. Define the *removeDevice* callback.

   This callback is invoked each time a device is removed. Class drivers should clean up the resources allocated for the device when it is attached through the *addDevice* routine.

3. Define the *suspendDevice* callback.

   This callback is invoked when the bus wants to put the device into a low power state.

4. Define the *resumeDevice* callback.

   This callback resumes the device from a low power state.

> Note: The operations in these routines should be done at a minimum; if there are any long time operations needed, the class driver should put these operations into a class driver specific handling task. This is because these routines are called in the Bus Monitoring task context (Configuring Bus Tasks on page 3), which is responsible for monitoring the whole USB bus of the underlying USB host controller. If these routines take too long, identification of other devices attached on the same bus is delayed, which adversely affects the user experience.
>
> Currently, the *suspendDevice* and *resumeDevice* do not have much to do, and so normally these routines are written as place holders.

Parent topic: USB Host Class Drivers Overview on page 23

Related information
USB Host Class Driver Registration on page 23

Setting the Device Type on page 24
Registering Vendor-Specific Devices on page 27
Using Standard Request Interfaces on page 27
Using Data Transfer Interfaces on page 29

1.4. Registering Vendor-Specific Devices

In some cases, a single driver can support devices that have similar functionality but different vendor IDs and product IDs.

About This Task

For example, several USB networking device vendors have made different device series to support USB networking functionality, with different vendor IDs and product IDs, but they can be handled by a single USB networking driver because they need to expose the same networking interface for VxWorks.

A driver can register a list of vendor devices in one registration call using **pDeviceList**, which is a pointer to an array of type **USBHST_DEVICE_ID**, which is defined in **usbHst.h**.

The array should be provided and populated by the class, or vendor-specific, driver that needs to support these devices specified in the array. For example, the USB networking driver uses this feature to register a list of supported devices in a batch. See **usb2EndInit()** in **usb2End.c**.

```
        pUsb2EndDriver->hstDriver.bFlagVendorSpecific = TRUE;
        pUsb2EndDriver->hstDriver.uVendorIDorClass = 0;
        pUsb2EndDriver->hstDriver.uProductIDorSubClass = 0;
        pUsb2EndDriver->hstDriver.uBCDUSBorProtocol = 0;
        pUsb2EndDriver->hstDriver.pDeviceList = &deviceIdList;

        pUsb2EndDriver->hstDriver.addDevice = usb2EndDeviceAdd;
        pUsb2EndDriver->hstDriver.removeDevice = usb2EndDeviceRemove;
        pUsb2EndDriver->hstDriver.suspendDevice = usb2EndSuspendResume;
        pUsb2EndDriver->hstDriver.resumeDevice = usb2EndSuspendResume;

    /* Register the client with USBD */

    USB2_END_VDBG ("usb2EndInit(): registering the client with USBD...\n",
                   1, 2, 3, 4, 5, 6);
    hstStatus = usbHstDriverRegister (&pUsb2EndDriver->hstDriver,
                                      NULL,
                                      "usbGen2Network");
```

Parent topic: USB Host Class Drivers Overview on page 23

Related information

USB Host Class Driver Registration on page 23
Setting the Device Type on page 24
Defining Event Callback Routines on page 26
Using Standard Request Interfaces on page 27
Using Data Transfer Interfaces on page 29

1.5. Using Standard Request Interfaces

The USB specification defines a set of standard requests, a subset of which must be supported by all USB devices.

About This Task

The USB request interface allows class drivers to issue standard USB requests or vendor-specific requests to the devices. Typically, a client uses these routines to interrogate a device's descriptors—thus determining the device's capabilities—and to set a particular device configuration. Some of the standard request interfaces exposed by the USBD are listed below. For examples using some of them, see the mouse class driver example.

usbHstGetDescriptor()

This routine is used to issue the **GET_DESCRIPTOR** USB standard request.

usbHstSetDescriptor()

This routine is used to issue the **SET_DESCRIPTOR** USB standard request.

usbHstGetInterface()

This routine is used to issue the **GET_INTERFACE** USB standard request.

usbHstSetInterface()

This routine is used to issue the **SET_INTERFACE** USB standard request.

usbHstGetConfiguration()

This routine is used to issue the **GET_CONFIGURATION** USB standard request.

usbHstSetConfiguration()

This routine is used to issue the **SET_CONFIGURATION** USB standard request.

usbHstClearFeature()

This routine is used to issue the **CLEAR_FEATURE** USB standard request.

usbHstSetFeature()

This routine is used to issue the **SET_FEATURE** USB standard request.

usbHstGetStatus()

This routine is used to issue the **GET_STATUS** USB standard request.

usbHstSetSynchFrame()

This routine is used to issue the **SYNCH_FRAME** USB standard request.

usbHstSetSel()

This routine issues the **SET_SEL** USB standard request.

Standard Request Interface for Setting the Descriptor

The following example shows the standard request interface **usbHstSetDescriptor()** provided by the USB host stack.

```
USBHST_STATUS IssueDeviceSetDescriptor
    (
    UINT32    hDevice,
    UINT32    uSize,
    PUCHAR    pBuffer
    )
```


WNDRVR

```
    {
    /* To store the setup packet */
    USBHST_SETUP_PACKET   SetupPacket;

    /* To store the USB request block */
    USBHST_URB            Urb;

    /* To store the usb status returned on submission of request */
    USBHST_STATUS         nStatus = USBHST_FAILURE;

    nStatus = usbHstSetDescriptor (hDevice, USBHST_DEVICE_DESC,
        0, 0, pBuffer, uSize);

    /* Check the status returned */
    if (USBHST_SUCCESS == nStatus)
        {
        /* Reset the device after set descriptor of device */
        }

    return nStatus;

    }
```

Parent topic: USB Host Class Drivers Overview on page 23

Related information

USB Host Class Driver Registration on page 23
Setting the Device Type on page 24
Defining Event Callback Routines on page 26
Registering Vendor-Specific Devices on page 27
Using Data Transfer Interfaces on page 29

1.6. Using Data Transfer Interfaces

Once a client has configured a device, the client can begin to exchange data with that device using the data transfer interfaces provided by the USBD.

About This Task

Control, bulk, interrupt, and isochronous transfers are described using a single **USBHST_URB** data structure (defined in **usbHst.h**). The data transfer interfaces exposed by the USBD are **usbHstURBSubmit()** and **usbHstURBCancel()**. Class drivers use these interfaces to:

- Submit a data transfer request.
- Cancel a data transfer request.
- Issue class-specific or vendor-specific requests using data transfers on the default control endpoint.

Data Transfer Interface

The following example shows the data transfer routine **usbHstURBSubmit()** provided by USB host stack.

```
/* Callback routine for blocking calls */

USBHST_STATUS Block_CompletionCallback(PUSBHST_URB pUrb)
```


WNDRVR

```
    {

    /* Check if pContext is valid */
    if ((NULL == pUrb) || (NULL == pUrb->pContext))
        {
        return USBHST_FAILURE;
        }

    /* Release the event(release a semaphore) */

    OS_RELEASE_EVENT((OS_EVENT_ID)pUrb->pContext);
    return USBHST_SUCCESS;

    } /* End of routine Block_CompletionCallback */


/* Issue block routine */

    USBHST_STATUS IssueBlockBulkSubmitUrb
    (
    UINT32    hDevice,
    UINT8     uRequest,
    UINT8     uRequestCode,
    UINT16    uValue,
    UINT16    uIndex,
    UINT32    uSize,
    PUCHAR    pBuffer
    )

    {
    /* To store the USB request block */
    USBHST_URB    Urb;

    /* To store the usb status returned on submission of request */
    USBHST_STATUS    nStatus = USBHST_FAILURE;
    /* To store the event id */
    OS_EVENT_ID    EventId;

    /* Allocate memory for urb and reset its values */
.........................
.........................
.........................

    /* Create transfer completion event for making a
     * blocking call (create a semaphore)
     */

    EventId = OS_CREATE_EVENT(OS_EVENT_NON_SIGNALED);

    /* Populate the Urb structure */
    USBHST_FILL_BULK_URB(&Urb, hDevice, 0, pBuffer, uSize,
        USBHST_SHORT_TRANSFER_OK, Block_CompletionCallback,
        NULL, USBHST_FAILURE);

    /* Call the submitURB routine to submit the URB. */
    nStatus = usbHstURBSubmit (&Urb);
```

```
    /* Check the status */
    if (USBHST_SUCCESS == nStatus)
        {

        /* Wait for the completion of the event(release of semaphore) */
        OS_WAIT_FOR_EVENT(EventId, OS_WAIT_INFINITE);

        /* Store the status returned by Urb */
        nStatus = Urb.nStatus;
        }

    /* Update parameters and free the resources */
.......................
.......................

    /* Destroy the event(destroy the semaphore) */
    OS_DESTROY_EVENT(EventId);

    /* Return the status of Urb */
    return nStatus;

    }
```

Parent topic: USB Host Class Drivers Overview on page 23

Related information

USB Host Class Driver Registration on page 23
Setting the Device Type on page 24
Defining Event Callback Routines on page 26
Registering Vendor-Specific Devices on page 27
Using Standard Request Interfaces on page 27

2. Human Interface Device Class

The USB Human Interface Device class (HID) package supports a single USB HID driver that supports the common functionality for all HID-based devices. This applies to host and function HID drivers.

Each HID subclass can be configured separately as necessary.

Creating a New HID Class Driver on page 31 This example shows how to create a new mouse class driver. The process can be applied to any HID class driver.

Keyboard, Mouse, and Printer Initialization on page 33 The keyboard, mouse, and printer drivers contain initialization routines that install standard open, close, read, write, and ioctl routines into the I/O system driver table. This allows an application to call these drivers using standard VxWorks system calls.

2.1. Creating a New HID Class Driver

This example shows how to create a new mouse class driver. The process can be applied to any HID class driver.

Procedure

1. Define the **USB_HID_REPORT_GET** in **usbhid.h**.

The **usbhid.h** header file is located in *installDir* **/pkgs_v2/connectivity/usb/host/class/hid/h**.

2. Modify the file.

   Change the following line in **usb2MseInit()** from **USB2_MSE_PROTOCOL** to **USBD_NOTIFY_ALL**:

```
pUsb2MseDriver->hstDriver.uBCDUSBorProtocol = USBD_NOTIFY_ALL;
```

3. Create a VSB project and add the mouse layer.

   Create a VSB project and include the mouse layer **USB_MOUSE** is enabled.

4. Build your VSB.
5. Create a VIP based on the VSB.
6. Include the appropriate components.

   Include the **INCLUDE_USB_GEN2_MOUSE**, **INCLUDE_USB_GEN2_MOUSE_INIT**, and host controller components. See Adding USB Mouse Device Support on page 36 and Support for the USB Host Stack on page 1.

7. Build the VIP.
8. Copy the **usb_hid_usages** file to your host default folder.

   Copy **usb_hid_usages** from *installDir* **\pkgs_v2\connectivity\usb\host\class\hid\h** to your host default folder. Type "ll" on the shell to locate this file in the list.

```
-> ll
build_vars_list
ctdt.c
ctdt.d
ctdt.o
dataSegPad.d
dataSegPad.o
linkSyms.d
linkSyms.o
partialImage.o
prjConfig.d
prjConfig.o
romfs_image_prj.d
romfs_image_prj.o
romfs_image_prj.s
sysALib.d
sysALib.o
sysLib.d
sysLib.o
usb_hid_usages    ===> Here it is.
usrAppInit.d
usrAppInit.o
usrEntry.d
usrEntry.o
usrRtpAppInit.d
usrRtpAppInit.o
version.d
version.o
vxWorks
vxWorks.sym
value = 0 = 0x0
```

9. Download the image and boot your machine.
10. Plug the HID device in and view the report descriptor.

    When the HID device is plugged into the host, it displays the report descriptor. For the mouse example, it displays output similar to the following:

```
Collection page=Generic_Desktop_ usage=Mouse_
Collection page=Generic_Desktop_ usage=Pointer_
Input   id=0 size=1 count=1 page=Button_ usage=1 Variable, logical range 0..1
Input   id=0 size=1 count=1 page=Button_ usage=2 Variable, logical range 0..1
Input   id=0 size=1 count=1 page=Button_ usage=3 Variable, logical range 0..1
Input   id=0 size=1 count=1 page=Button_ usage=4 Variable, logical range 0..1
Input   id=0 size=1 count=1 page=Button_ usage=5 Variable, logical range 0..1
Input   id=0 size=1 count=1 page=Button_ usage=6 Variable, logical range 0..1
Input   id=0 size=1 count=1 page=Button_ usage=7 Variable, logical range 0..1
Input   id=0 size=1 count=1 page=Button_ usage=8 Variable, logical range 0..1
Input   id=0 size=8 count=1 page=Generic_Desktop_ usage=X_ Variable Relative, logical range -12
7..12
7
Input   id=0 size=8 count=1 page=Generic_Desktop_ usage=Y_ Variable Relative, logical range -12
7..12
7
Input   id=0 size=8 count=1 page=Generic_Desktop_ usage=Wheel_ Variable Relative, logical rang
e -127
..127
Input   id=0 size=8 count=1 page=Consumer_ usage=AC_Pan_ Variable Relative, logical range -127.
.127
End collection
End collection
```

11. Create a structure from the HID information.

    Parse the HID report item and create the report structure for this HID device. Thus, from the example log message, the first byte shows button status, the second byte is the X position, and third byte is the Y position. The mouse report structure would be:

```
typedef struct usb2_mse_report
    {
    UINT8 buttonState;              /* Button State */
    INT8 xMove;                     /* signed x-displacement */
    INT8 yMove;                     /* signed y-displacement */
    } __attribute__((packed)) USB2_MSE_REPORT;
```

    Use this report structure to parse the data from the device and pass the item information to the application.

Parent topic: Human Interface Device Class on page 31

Related information

Keyboard, Mouse, and Printer Initialization on page 33

2.2. Keyboard, Mouse, and Printer Initialization

The keyboard, mouse, and printer drivers contain initialization routines that install standard open, close, read, write, and ioctl routines into the I/O system driver table. This allows an application to call these drivers using standard VxWorks system calls.

WNDRVR

Note: Currently, the USB keyboard and mouse devices support two mechanisms, the evdev system and a legacy mode. The information in this topic applies only to legacy mode.

For example, an application might want to monitor a keyboard's input. First, the application opens the keyboard with the system call to **open()**:

```
fileDescr = open ("/usb2Kbd/0 ", 2, 0);
```

The application can now call the system's **read()** routine in a loop:

```
while (read (fileDescr, &inChar, 1) != 1);
```

These operations can be used for the mouse, printer, and speaker drivers as well.

Note: For information on using the evdev framework with USB devices, see the *VxWorks 7 Event Device Framework User's Guide*.

Parent topic: Human Interface Device Class on page 31

Related information

Creating a New HID Class Driver on page 31

3. Adding USB Keyboard Device Support

To add support for the USB keyboard, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the a USB keyboard device, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the USB keyboard to the VSB.

   To configure a VxWorks Source Build (VSB) project with support for the keyboard class driver, include the following VSB configuration options:

   - **USB_CLASS**
   - **USB_HELPER**
   - **USB_KEYBOARD**

For more information, see USB Host Class Drivers Overview on page 23.

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1.

4. Add support for the USB keyboard to the VIP.

   Additionally include the **INCLUDE_USB_GEN2_KEYBOARD** component to the VIP.

5. Configure the keyboard.

   Set the **INCLUDE_USB_GEN2_KEYBOARD** configuration parameters, as needed. For more information on these parameters and their default values, see the *Wind River USB Component Reference*.

6. Optionally add support for initialization at startup.

   Additionally include the **INCLUDE_USB_GEN2_KEYBOARD_INIT** component to your VIP to add support for initialization at startup. For details, see Initializing the USBD on page 5.

7. Build the VIP.

Adapting a USB Keyboard Driver to the evdev System on page 35 Currently, the USB keyboard driver supports two models: a legacy model and the evdev model.
Using Multi-Language Keyboard Support and Hot Keys on page 36 You can choose to add support to modify the keyboard layout on targets to take into account the national versions of keyboard layouts.

3.1. Adapting a USB Keyboard Driver to the evdev System

Currently, the USB keyboard driver supports two models: a legacy model and the evdev model.

About This Task

The legacy model is the same as VxWorks 6.9.x. The evdev model adapts the USB keyboard driver to the input framework, by using the keyboard library based on the evdev core driver and library.

If the evdev layer **EVDEV_LIB** is added, then the evdev model of USB keyboard driver is loaded, otherwise, the legacy model of USB keyboard driver loaded. For example, to use the evdev model:

Procedure

1. Create a VSB project.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the USB keyboard to the VSB.

   Add the **USB_KEYBOARD** keyboard layer to your VSB project.

3. Add the **EVDEV_LIB** evdev keyboard library layer to your VSB project.
4. Build these two layers.
5. Create a VIP based on the VSB.
6. Add the **INCLUDE_USB_GEN2_KEYBOARD USB** keyboard component to your VIP.
7. Optionally, add the initialization component.

Add the **INCLUDE_USB_GEN2_KEYBOARD_INIT USB** keyboard initialization component to your VIP if you wan initialization on startup. For debug mode, this component is optional.

8. Add the **INCLUDE_EVDEV_LIB_KBD** evdev keyboard library component to your VIP project.
9. Build VIP project.

Parent topic: Adding USB Keyboard Device Support on page 34

Related information
Using Multi-Language Keyboard Support and Hot Keys on page 36

3.2. Using Multi-Language Keyboard Support and Hot Keys

You can choose to add support to modify the keyboard layout on targets to take into account the national versions of keyboard layouts.

About This Task

As an example of using national versions of keyboard layouts, the keystroke "[" ";" "" on a US keyboard maps to "Ü" "Ö" "Ä" on a German keyboard. To create a new country layout, follow these steps.

Procedure

1. Create the map file for the proper country.

   To create a new map file, refer to the **evdevLibKbdMapUS.c**.

   Note: The countryCode in structure **EV_DEV_KBD_MAP** is the Language Identifiers (LANGIDs). For all human interface device and LANGID information, refer to https://usb.org/hid. For the latest LANGID definitions, go to https://docs.microsoft.com/en-us/windows/win32/intl/language-identifier-constants-and-strings.

2. Create the component for the proper country.

   To create a new component, refer to the **41evdevLibKbdMapUS.cdf**.

3. Set the Language Identifiers for the keyboard.

   Language Indentifiers are set with the **EV_DEV_IO_SET_COUNTRY_CODE** ioctl operation. The default LANGID is **EV_DEV_KBD_LANG_ENGLISH_US**.

Parent topic: Adding USB Keyboard Device Support on page 34

Related information
Adapting a USB Keyboard Driver to the evdev System on page 35

4. Adding USB Mouse Device Support

To add support for the USB mouse, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for a USB mouse device, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the USB mouse to the VSB.

   To configure a VxWorks Source Build (VSB) project with support for the mouse class driver, include the following VSB configuration options:

   - **USB_CLASS**
   - **USB_HELPER**
   - **USB_MOUSE**

   For more information, see USB Host Class Drivers Overview on page 23.

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Add support for the USB mouse to the VIP.

   Additionally include the **INCLUDE_USB_GEN2_MOUSE** component to the VIP.

5. Configure the mouse.

   Set the **INCLUDE_USB_GEN2_MOUSE** configuration parameters, as needed.

6. Optionally add support for initialization at startup.

   Additionally include the **INCLUDE_USB_GEN2_MOUSE_INIT** component to your VIP to add support for initialization at startup. For details, see Initializing the USBD on page 5.

7. Configure the Select feature for GEN2 mouse class drivers.

   The select feature allows tasks to pend on the drivers while waiting for I/O requests from devices. When the devices are ready to send data, the driver notifies the tasks.

   To include support for the USB mouse class driver select feature, include the **INCLUDE_SELECT** parameter.

   To support the select feature, the mouse drivers must be set to the interrupt mode. Do this by issuing an ioctl request, with the request option as **SIO_MODE_INT**.

5. Adding USB Touchscreen Device Support

To add support for the USB touchscreen, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the a USB touchscreen device, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the USB touchscreen to the VSB.

   To configure a VxWorks Source Build (VSB) project with support for the touchscreen class driver, include the following VSB configuration options:

   - **EVDEV_LIB**
   - **USB_TOUCHSCREEN**

3. Build these two layers.
4. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

5. Add the **INCLUDE_USB_TOUCH_SCREEN** USB touchscreen component to the VIP.
6. Optionally add support for initialization at startup.

   Additionally include the **INCLUDE_USB_TOUCH_SCREEN_INIT** component to your VIP to add support for initialization at startup. For details, see Initializing the USBD on page 5.

7. Build the VIP.

6. Adding USB Printer Device Support

To add support for the USB printer, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the a USB printer device, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the USB printer to the VSB.

   To configure a VxWorks Source Build (VSB) project with support for the printer class driver, include the following VSB configuration options:

   - **USB_CLASS**
   - **USB_HELPER**
   - **USB_PRINTER**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Add the **INCLUDE_USB_GEN2_PRINTER** USB printer component to the VIP.
5. Optionally add support for initialization at startup.

   Additionally include the **INCLUDE_USB_GEN2_PRINTER_INIT** component to your VIP to add support for initialization at startup. For details, see Initializing the USBD on page 5.

6. Build the VIP.

7. UAS/UASP Mass Storage Class

Wind River USB for VxWorks 7 provides limited support for USB Attached SCSI protocol (UAS/UASP) Mass Storage Class devices.

The UASP protocol supports both USB 2.0 and USB 3.0. In USB 2.0, there is no Bulk Streams support, so the protocol is effectively sequential based. The USB 3.0 adds Bulk Streams support, which can provide asynchronous completion of different transfers.

UAS/UASP Mass Storage Class devices use a four pipe model. The protocol does not mix commands, status, and data on a single pipe. It also allows for vendor-specific pipes.

Adding USB Mass Storage Device Support on page 39 To add support for a USB mass storage device, you must include additional components in your VSB and VIP projects.
USB GEN2 Mass Storage Quirky Device Handling on page 41 Some Bulk-Only mass storage devices, which are not fully compatible with the USB mass storage class specification or the related Command Set Specification, are called "quirky devices" in this documentation and in the driver sources.

7.1. Adding USB Mass Storage Device Support

To add support for a USB mass storage device, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

Note: Some components include compiler macros and flags that are defined in the makefile in the *installDir* **/pkgs_v2/connectivity/usb/target** directory.

The USB peripheral stack includes the following configuration option for rebuilding source, **ADDED_CFLAGS+=NET2280_DMA_SUPPORTED**, which enables DMA transfer for the NET 2280 driver (currently the ISP 1582 driver does not support DMA transfer).

To add support for the a USB mass storage device, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for a USB mass storage device to the VSB.

   To configure a VxWorks Source Build (VSB) project with support for the printer class driver, include the following VSB configuration options:

   - **USB_CLASS**
   - **USB_HELPER**
   - **USB_STORAGE**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Add the **INCLUDE_USB_GEN2_STORAGE** USB mass storage component to the VIP.
5. Configure the mass storage device.

   Set the **INCLUDE_USB_GEN2_STORAGE** configuration parameter values, as needed.

6. Optionally add support for initialization at startup.

   Additionally include the **INCLUDE_USB_GEN2_STORAGE_INIT** component to your VIP to add support for initialization at startup. For details, see Initializing the USBD on page 5.

7. Configure the initialization component.

   Set the **INCLUDE_USB_GEN2_STORAGE_INIT** configuration parameter values, as needed.

8. Include a file system.
9. Optionally, include utility components.
10. Build the VIP.

Parent topic: UAS/UASP Mass Storage Class on page 39

Related information

USB GEN2 Mass Storage Quirky Device Handling on page 41

7.2. USB GEN2 Mass Storage Quirky Device Handling

Some Bulk-Only mass storage devices, which are not fully compatible with the USB mass storage class specification or the related Command Set Specification, are called "quirky devices" in this documentation and in the driver sources.

The GEN2 mass storage class driver provides a quirky device handling mechanism, by which some of the devices that are in violation of the specification can apply a workaround to work properly with VxWorks. The mechanism is based on device ID matching to handle the quirky devices.

If a quirky device requires a workaround, a quirky device information entry can be made in the user configuration module of the driver (**usrUsb2MscInit.c**), specifying the VID/PID/REV range of the device, along with any bitwise ORed flags that describe the required workaround.

The driver has pre-defined quirky flags. The quirky device information array has only one entry, which describes one really quirky device, where it reports VID/PID/REV all as zero. That device can work with this driver without any quirky flags, thus, that entry is filled for demonstration purposes.

The following is a list of the pre-defined quirky device flags, which can be found in *installDir* **/pkgs_v2/connectivity/usb/host/class/storage/h/usb2Msc.h**.

```
/* Define some known device quirky flags */

        #define USB2_MSC_DEV_FLAG_ODD_SECTORS                   (1ULL << 0)
        #define USB2_MSC_DEV_FLAG_IGNORE_DATA_RESIDUE           (1ULL << 1)
        #define USB2_MSC_DEV_FLAG_MAX_64_BLOCKS_TRANSFER        (1ULL << 2)
        #define USB2_MSC_DEV_FLAG_NO_MS_WP_CHECK                (1ULL << 3)
        #define USB2_MSC_DEV_FLAG_SINGLE_LUN                    (1ULL << 4)
        #define USB2_MSC_DEV_FLAG_IGNORE_CBI_INTERRUPT          (1ULL << 5)
        #define USB2_MSC_DEV_FLAG_UNABLE_PREVENT_REMOVAL        (1ULL << 6)
        #define USB2_MSC_DEV_FLAG_UNMACTHED_CSW_TAG             (1ULL << 7)
        #define USB2_MSC_DEV_FLAG_DELAY_AFTER_CBW               (1ULL << 8)
        #define USB2_MSC_DEV_FLAG_BBB_USE_32_BYTES_CBW          (1ULL << 9)
        #define USB2_MSC_DEV_FLAG_DUMMY_INQUIRY                 (1ULL << 10)
        #define USB2_MSC_DEV_FLAG_IGNORE_DEVICE                 (1ULL << 11)
```

These values are grouped into three groups:

USB2_MSC_DEV_FLAG_STRICT for: no flags

USB2_MSC_DEV_FLAG_RELAXED for:

USB2_MSC_DEV_FLAG_ODD_SECTORS
USB2_MSC_DEV_FLAG_IGNORE_DATA_RESIDUE
USB2_MSC_DEV_FLAG_MAX_64_BLOCKS_TRANSFER
USB2_MSC_DEV_FLAG_NO_MS_WP_CHECK
USB2_MSC_DEV_FLAG_SINGLE_LUN
USB2_MSC_DEV_FLAG_IGNORE_CBI_INTERRUPT
USB2_MSC_DEV_FLAG_UNABLE_PREVENT_REMOVAL

USB2_MSC_DEV_FLAG_ALL for:

USB2_MSC_DEV_FLAG_ODD_SECTORS
USB2_MSC_DEV_FLAG_IGNORE_DATA_RESIDUE
USB2_MSC_DEV_FLAG_MAX_64_BLOCKS_TRANSFER
USB2_MSC_DEV_FLAG_NO_MS_WP_CHECK
USB2_MSC_DEV_FLAG_SINGLE_LUN
USB2_MSC_DEV_FLAG_IGNORE_CBI_INTERRUPT
USB2_MSC_DEV_FLAG_UNABLE_PREVENT_REMOVAL
USB2_MSC_DEV_FLAG_UNMACTHED_CSW_TAG
USB2_MSC_DEV_FLAG_DELAY_AFTER_CBW
USB2_MSC_DEV_FLAG_BBB_USE_32_BYTES_CBW
USB2_MSC_DEV_FLAG_DUMMY_INQUIRY

These three groups are used for the parameter USB2_MSC_QUIRKY_DEV_SUPPORT_LEVEL.

If a device is found to be in accordance with any of the above flags, the device entry can be added in the **usrUsb2MscQuirkyDevices[ ]** array, found in the GEN2 mass storage initialization source file located at *installDir* **/pkgs_v2/connectivity/usb/host/class/storage/cfg/usrUsb2MscInit.c**, as shown in the following code sample:

```
/* User configurable quirky device table */

LOCAL USB2_MSC_QUIRKY_INFO usrUsb2MscQuirkyDevices[] =
    {
    /*
     * EAGET CM981 reports VID/PID/REV all as zero,
     * but it works fine without any flags; This entry
     * is just to log this device as an demonstration for
     * quirky device entry usage.
     */

    USB2_MSC_QUIRKY(0x0000,0x0000,0x0000,0x0000,
        "EAGET CM981/E981HL 4GB",
        USB2_MSC_SUBCLASS_RESERVED,
        USB2_MSC_PROTOCOL_RESERVED,
        NULL,NULL,NULL,
        0
        )
    };
```

Note: Changing the initialization source file does not require re-compiling the driver source, but it does require re-compiling the VxWorks image.

USB2_MSC_QUIRKY_INFO and **USB2_MSC_QUIRKY( )** used in the above code are defined in *installDir* **/pkgs_v2/connectivity/usb/host/class/storage/h/usb2Msc.h**.

```
typedef void (*USB2_MSC_SETUP_FUNCPTR)
    (
    UINT32 hDevice,         /* Device handle for the device */
    void *pQuirky           /* Pointer to quirky info below */
    );

typedef STATUS (*USB2_MSC_MOUNT_FUNCPTR)
    (
    struct usb2_msc_device * pMscDevice    /* device for mounting */
    );

typedef STATUS (*USB2_MSC_UNMOUNT_FUNCPTR)
    (
    struct usb2_msc_device * pMscDevice    /* device for unmounting */
    );

typedef struct usb2MscPrivateFuncs
    {
    USB2_MSC_SETUP_FUNCPTR setup;   /* setup function ptr */
    USB2_MSC_MOUNT_FUNCPTR mount;   /* mount function ptr */
    USB2_MSC_UNMOUNT_FUNCPTR unmount; /* unmount function ptr */
    } USB2_MSC_PRIVATE_FUNCS;

typedef struct usb2_msc_quirky_info
    {
    USBHST_DEVICE_ID devId;         /* Device ID info for match */
    const char * deviceName;        /* Quirky device name */
    UINT8 forceSubclass;            /* force sub class code */
    UINT8 forceProtocol;            /* force protocol code */
    UINT64 quirkyFlag;              /* quirky flags */
    USB2_MSC_PRIVATE_FUNCS funcs;   /* private function pointers */
    }USB2_MSC_QUIRKY_INFO;

#define USB2_MSC_QUIRKY(vendorId,productId,driverBCDLow,driverBCDHigh,  \
            deviceName, subclass,protocol,setup,mount,unmount,flags)    \
    {                                                                   \
        {                                                               \
        {NULL}, 0,                                                      \
        vendorId, productId, driverBCDLow, driverBCDHigh, 0,            \
        0, 0, 0,                                                        \
        0, 0, 0,                                                        \
        USBD_MATCH_ID_VENDORID | USBD_MATCH_ID_PRODUCTID,               \
        NULL                                                            \
        },                                                              \
    deviceName,                                                         \
    subclass,                                                           \
    protocol,                                                           \
    flags,                                                              \
    {setup, mount, unmount},                                            \
    }
```

The parameters for USB2_MSC_QUIRKY( ) are defined below.

- vendorId and productId correspond to the idVendor and idProduct fields in the Device Descriptor of the device.

- bcdDeviceLow and bcdDeviceHigh correspond to the bcdDevice field in the Device Descriptor of the device, and form a range of the device revisions from bcdDeviceLow (inclusive) to bcdDeviceHigh (inclusive), that have the quirky behavior specified by the flags field in this entry.
- deviceName corresponds to a constant string of the device name description that has such a quirky behavior as specified by the flags field in this entry.
- forceSubclass and forceProtocol are for a case where the quirky mass storage device reports the wrong bInterfaceSubClass or bInterfaceProtocol in its Interface Descriptor. In this case, if the driver still uses the wrong bInterfaceSubClass or bInterfaceProtocol, then the driver issued Command Set may not be the expected Command Set from the device, and thus will not work properly. To ensure the driver can still work with such a device, a quirky device entry can specify the corrected bInterfaceSubClass or
- bInterfaceProtocol in the forceSubclass or forceProtocol, respectively. If the bInterfaceSubClass or bInterfaceProtocol reported by the device is correct, then these fields can be set to be USB2_MSC_SUBCLASS_RESERVED, which will make the driver keep using the device reported bInterfaceSubClass and bInterfaceProtocol in the Interface Descriptor.
- setup can be specified by the entry as a function pointer to a customized device initialization routine. This is for the case where a quirky device must be issued some customized commands before normal mass storage operation can start. The USB2_MSC_SETUP_FUNCPTR takes two parameters: one parameter is the hDevice handle, which can be used to issue any requests provided by the USBD API; another parameter is the quirky device entry itself, to allow further customization in the customized initialization routine.
- mount can be specified by the entry as a function pointer to a customized device mounting routine. This is for the case where a quirky device must be issued some customized commands for mounting, reading, and writing. The USB2_MSC_MOUNT_FUNCPTR takes one parameter pMscDevice as a pointer to a storage device for mounting. In this mounting function, you can customize the private reading or writing function if needed.
- unmount can be specified by the entry as a function pointer to a customized device unmounting routine. This is for the case where a quirky device must be issued some customized commands for mounting a quirky device. The USB2_MSC_MOUNT_FUNCPTR takes one parameter pMscDevice as a pointer to a storage device for unmounting.
- flags is a bitwise ORed flag variable of the pre-defined flag macros. This specifies the quirky behavior(s) for which workaround(s) for this device will be applied. The flags are as follows:

  USB2_MSC_DEV_FLAG_ODD_SECTORS

  Normally, a USB stick or any other storage device may report its total sector number as an even number, but some devices report an odd number. In this case, it is better to subtract one from the sector number and report it. Typically, a smaller number is better than a larger number. This flag will subtract one sector from the total sectors if the device reports an odd total sector number.

  USB2_MSC_DEV_FLAG_IGNORE_DATA_RESIDUE

  Some quirky devices may report wrongly computed **dCSWDataResidue**, which the specification states as:

  “For Data-Out the device shall report in the **dCSWDataResidue** the difference between the amount of data expected as stated in the **dCBWDataTransferLength**, and the actual amount of data processed by the device."

  "For Data-In the device shall report in the **dCSWDataResidue** the difference between the amount of data expected as stated in the **dCBWDataTransferLength** and the actual amount of relevant data sent by the device. The **dCSWDataResidue** shall not exceed the value sent in the **dCBWDataTransferLength**.”

  If wrongly computed **dCSWDataResidue** is returned, we may try to deduce the actual residue, based on the transfer status and actual size transferred.

  You can check the returned **dCSWDataResidue** for **INQUERY** and **READ_CAPCITY**. If the **bCSWStatus** is **GOOD** and the returned size is as expected (but **dCSWDataResidue** is wrongly computed), we may set a flag to ignore **dCSWDataResidue** for all other commands.

  USB2_MSC_DEV_FLAG_MAX_64_BLOCKS_TRANSFER

  Some quirky devices can not transfer more than 64 sectors one time. This may be a hardware limitation issue. Since one sector is 512 bytes at least, we consider it as “do not transfer 32KB one time.” This flag has no effect with standard devices, because fewer sectors for one transfer will not cause any error. This flag may affect performance for very large data block.

  This flag has higher priority than the parameter USB2_MSC_MAX_TRANSFER_BLOCKS.

WNDRVR

Storage checks USB2_MSC_MAX_TRANSFER_BLOCKS first, then checks this flag; if this flag is set, the driver will transfer 64 sectors at most.

USB2_MSC_DEV_FLAG_NO_MS_WP_CHECK

If the driver configuration flag USB2_MSC_WRITE_PROTECTION_CHECK_ENABLE is set to TRUE, the driver will try to use the Mode Sense command to frequently check the Write Protection status of the device.

However, not all devices support this command. For such devices, they may not handle Mode Sense correctly. So if the Write Protection check still needs to be enabled but has to work with such a device, the USB2_MSC_DEV_FLAG_NO_MS_WP_CHECK can be set in a quirky device entry.

USB2_MSC_DEV_FLAG_SINGLE

Some quirky devices can not support GetLun request; most of these device are Card Readers or CBI devices. This flag prevents the driver from sending a GetLun request to these devices and simply regards them as a single LUN (logical units) device. This feature is incompatible with standard devices and can make standard devices work incorrectly.

If the device cannot reply to the **GET_MAX_LUN** command or replies with an invalid LUN number, the driver considers this device to be a single LUN device.

USB2_MSC_DEV_FLAG_IGNORE_CBI_INTERRUPT

Some quirky devices report to be Control/Bulk/Interrupt transport protocol in the Interface Descriptor, and also have an Interrupt In endpoint, but such devices do not really report the Status of the Command through the Interrupt In endpoint, rather, they report Status information through the default Control endpoint. For such quirky devices, if the driver still tries to check the Interrupt In endpoint for Status information, the driver may become very slow, because it has to wait for the request time-out.

USB2_MSC_DEV_FLAG_UNABLE_PREVENT_REMOVAL

Some quirky devices can respond to a SCSI command **USB_SCSI_PREVENT_REMOVAL**. This flag can prevent driver sending this command. Since this command is not used in current driver, this flag is not used.

USB2_MSC_DEV_FLAG_UNMACTHED_CSW

Some quirky devices may report an invalid CSW (command status wrapper) Signature.

If the two flags do not match, it should be treated as a transfer error as the specification defines.

The driver ignores an invalid CSW if this flag is set.

USB2_MSC_DEV_FLAG_DELAY_AFTER_CBW

Some quirky devices may need a little delay just after the CBW transfer, so you may need to add a frame of time delay here. This may be due to slow firmware in the device side in that it needs some time to interpret the CBW and transition to the proper next stage in its device state machine. Note that we have not encountered such a device in testing, and it may introduce a bit of a performance penalty if we add a delay here. If you encounter such a quirky device, specify an entry in the quirky device list for the device with the USB2_MSC_DEV_FLAG_DELAY_AFTER_CBW flag.

USB2_MSC_DEV_FLAG_BBB_USE_32_BYTES_CBW

For normal Bulk-Only transport protocol devices, the CBW (Command Block Wrapper) is 31 bytes in size.

However, some quirky devices may need 32 bytes in CBW transfer, with the last byte (byte 32) as a trailing zero. For such devices, if a normal 31 bytes CBW is sent to them, they cannot interpret the Command correctly and thus the driver can not work with them.

USB2_MSC_DEV_FLAG_DUMMY_INQUIRY

Some quirky devices do not respond to a data request inquiry or respond with incorrect data. Setting this flag makes the driver fill a dummy inquiry data for these devices. Since this dummy inquiry data may not suit all quirky devices, this flag may not work well with all quirky devices.

USB2_MSC_DEV_FLAG_IGNORE_DEVICE

Some customized devices may report to be USB mass storage devices in their Interface Descriptor, but the devices may need to be handled differently with a customized mechanism. For such devices, they may not be taken by the GEN2 mass storage class driver to be USB mass storage devices, so a USB2_MSC_DEV_FLAG_IGNORE_DEVICE flag in its quirky device entry needs to be set. This will make the GEN2 mass storage class driver ignore this device in the early stage of the device adding procedure.

WNDRVR

Parent topic: UAS/UASP Mass Storage Class on page 39

Related information
Adding USB Mass Storage Device Support on page 39

8. Adding USB Network Driver Support

To add support for the USB network class driver, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the a USB network driver, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the USB network drivver to the VSB.

   To configure a VxWorks Source Build (VSB) project with support for the printer class driver, include the following VSB configuration options:

   - **USB_CLASS**
   - **USB_HELPER**
   - **USB_NETWORK**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Add support for the USB network class drivers to the VIP.

   Additionally, include one of the following components, as appropriate for your chipset:

   - **INCLUDE_USB_GEN2_PEGASUS**
   - **INCLUDE_USB_GEN2_DM960X**
   - **INCLUDE_USB_GEN2_ASIX**

5. Optionally add support for initialization at startup.

   Additionally include the **INCLUDE_USB_GEN2_END_INIT** component to your VIP to add support for initialization at startup. For details, see Initializing the USBD on page 5.

6. Configure the network class drivers.

   Set the **INCLUDE_USB_GEN2_END_INIT** configuration parameter value, as needed.

7. Include networking support.
    1. To include USB END driver initialization, include network initialization.
    2. Depending upon system requirements, increase the IP_MAX_UNITS value.

    For example, in a system that initializes an FEI Ethernet interface and a USB Pegasus Ethernet interface, the IP_MAX_UNITS must be at least 2 (in VxWorks 5.5, the default value is 1. For VxWorks 6.0, the default value is 4). If you want to test the Ethernet device, you must also include the network show routines component.

    3. For Pegasus interface support, include the **INCLUDE_IFCONFIG** component.
    4. Set the IP address of the USB END device with parameter USB_GEN2_END_IP_ADDRESS. Do not configure parameters with **IFCONFIG_*** (where * is a number) to avoid having multiple IPs on the USB END device.
8. Build the VIP.

9. Adding USB Serial Class Driver Support

To add support for the USB serial class driver, you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for the a USB printer device, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for the USB serial class driver to the VSB.

   To configure a VxWorks Source Build (VSB) project with support for the printer class driver, include the following VSB configuration options:

   - **USB_CLASS**
   - **USB_HELPER**
   - **USB_SERIAL**
3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Add the USB serial class driver components **INCLUDE_USB_GEN2_SER_FTDI232** and **INCLUDE_USB_GEN2_SER_PL2303** to the VIP. **INCLUDE_USB_GEN2_SER_FTDI232** component provides support for the USB GEN2 FTDI 232 Serial Adapter and **INCLUDE_USB_GEN2_SER_PL2303** component provides support for USB Gen2 Prolific 2303 Adapter.
5. Add support for initialization at startup.

   Additionally include the **INCLUDE_USB_GEN2_SERIAL_INIT** component to your VIP to add support for initialization at startup. This is the initialization component for the USB-to-serial adapter device and provides initialization at startup. For details, see Initializing the USBD on page 5.

6. Set the **INCLUDE_USB_GEN2_SERIAL_INIT** configuration parameter values, as needed.
7. Include the **INCLUDE_USB_GEN2_SERIAL_PCCONSOLE_INIT** component.

   This is the initialization component for the serial SIO device and provides initialization at startup. For details, see Initializing the USBD on page 5.

   The USB host console has two parts—one part is the driver for the adapter; the other part is the driver for the serial device. Therefore, two initialization components are required.

8. Set the **INCLUDE_USB_GEN2_SERIAL_PCCONSOLE_INIT** configuration parameter values, as needed.

   The default values for these configuration parameters should typically not be changed.

9. Include the **INCLUDE_USB_GEN2_HELPER** component.

   This component provides a set of support services common to all GEN2 class drivers.

10. Set the **INCLUDE_USB_GEN2_HELPER** configuration parameter values, as needed.
11. Build the VIP.

PC Console Parameter Values on page 48 The default values for the **INCLUDE_USB_GEN2_SERIAL_PCCONSOLE_INIT** component configuration parameters should typically not be changed.

9.1. PC Console Parameter Values

The default values for the **INCLUDE_USB_GEN2_SERIAL_PCCONSOLE_INIT** component configuration parameters should typically not be changed.

INCLUDE_USB_GEN2_SERIAL_PCCONSOLE_INIT Default Values

USB_GEN2_SERIAL_CONSOLE_DEVICE_NAME

This parameter defines the host port console name. The default is **/usbPcCon/**. This name is registered to VxBus.

USB_GEN2_SERIAL_PCCON_ATTACH_DEVICE_NAME

This parameter defines the name of the attached USB GEN2 serial device. The default setting is **/usb2ttyS/**. You can use the **devs** shell command to display this device name.

USB_GEN2_SERIAL_PCCON_DEFAULT_BAUDRATE

This parameter defines the Serial PC Console Baud Rate. The default setting is **9600**. This setting may be overwritten by the console's baudrate setting.

USB_GEN2_SERIAL_PCCON_DUMMY_BUFFER_SIZE

This parameter sets the USB GEN2 Serial PC Console Dummy Buffer Size. The default size is **4096**.

This buffer is used for output when there is no actual USB-to-Serial adapter attached. In such a case, output print data will be sent to the dummy buffer. The dummy buffer data would be flushed to the real device when the real device is attached.

This is used to save the output data at VxWorks startup. Because the USB stack may initialize very late, any message output before USB initializes can be written to the dummy buffer, and the user will not lose any output messages.

USB_GEN2_SERIAL_PCCON_OUT_BUFFER_SIZE

This parameter defines the USB GEN2 Serial PC Console Out Buffer Size. It must be larger than the TTY LIB Buffer (whose default setting is 512). The shell and other tasks that use **ttylib** for IO will pick up data from this buffer.

USB_GEN2_SERIAL_PCCON_TASK_PRIORITY

This parameter defines the USB GEN2 Serial PC Console TTY Task Priority. The default setting is **100**.

A high task priority can make IO run more smoothly and the console able to respond faster to user commands. For example, pressing CTRL+C in the console gives an interrupt to the shell, If this task priority is very high, CTRL+C will be sent to shell task faster. This parameter is most effective when the CPU is very busy; when the CPU is not very busy, the priority setting has little effect.

USB_GEN2_SER_MAX_CONSOLE

This parameter sets the maximum number of USB-to-serial port host consoles. The default setting is **1**. The maximum number supported is 10.

USB can supply more than one console to the user. For example, it can provide one console for the default console and another for serial access.

Default Console TTY Parameters

To use the USB console as the default console TTY, you must change the settings for the **CONSOLE_NAME** parameter.

CONSOLE_NAME

The CONSOLE_NAME parameter defines which device is used as the default console. You must set this parameter to **/ttyUSB0** to use the first USB virtual serial console as the default console of VxWorks.

Parent topic: Adding USB Serial Class Driver Support on page 47

10. Adding USB Video Device Support

10.1. About the USB Video Class Driver

The USB Video Class (UVC) driver can make VxWorks communicate with USB Video Class devices and retrieve information about their capabilities and characteristics. It can also negotiate the desired settings of the video stream such that an application may successfully control the device and retrieve the desired video data from it.

The VxWorks UVC driver follows the specification and aims to be compatible with as many devices as possible. The following devices are supported:

- Canyon
- Logitech C920
- Intel RealSense Camera R200

Only the following image format types are supported:

- Uncompressed payload
- MJPEG payload
- Frame-based payload

10.2. Adding USB Video Class Support

To add support for the USB video class (UVC), you must include additional components in your VSB and VIP projects.

About This Task

For details on configuring a VxWorks image project (VIP), see the VxWorks 7 Configuration and Build Guide. For component and parameter descriptions, see the *Wind River USB Component Reference*.

To add support for UVC, include the appropriate components in both your VxWorks source build (VSB) and your VxWorks image project (VIP).

Procedure

1. Create a VSB.

   Create a VSB according to the steps in Create the VxWorks Source Build on page 1.

2. Add support for UVC to the VSB.

   To configure a VxWorks Source Build (VSB) project with support for UVC, include the following VSB configuration options:

   - **USB**
   - **USB_CORE**
   - **USB_CTLR**
   - **USB_CCORE**
   - **USB_EHCI**
   - **USB_XHCI**
   - **USB_HOST**
   - **USB_CLASS**
   - **USB_HELPER**
   - **USB_UVC**
   - **USB_HCORE**
   - **USB_PHY**

3. Create a VIP based on the VSB.

   Create a VIP according to the steps in Create the VxWorks Image Project on page 1

4. Add support for UVC to the VIP.

   To configure a VxWorks image project (VIP) with support for UVC, include the following components in the VIP:

   - **INCLUDE_USB**
   - **INCLUDE_USB_INIT**
   - **INCLUDE_EHCI_INIT**
   - **INCLUDE_USB_XHCI_HCD_INIT**
   - **INCLUDE_USB_GEN2_VIDEO_INIT**
   - **INCLUDE_USB_GEN2_HELPER**
   - **INCLUDE_USB_GEN2_VIDEO**
   - **INCLUDE_USB_HOST_CLASS_INIT**

5. Enable the display of UVC driver debug messages. This step will configure the UVC driver to print debug messages for the UVC device initialization phase and for all the activities.

Add the **INCLUDE_USB_TRACK_GEN2_VIDEO** component to your VIP.

6. Set the **USB_TRACK_GEN2_VIDEO** parameter value. This is the default value for the video class device debug-level bitmap.

   The bit fields are defined as follows:

Bit 0:	INFO	(0x01)
Bit 1:	ERROR	(0x02)
Bit 2:	WARNING	(0x04)
Bit 3:	NORMAL	(0x08)
Bit 4:	VERBOSE	(0x10)

7. Build your VIP.

10.3. Coding UVC Applications

About This Task

Coding applications can operate the video device with open( ), read( ), close( ), select( ), and ioctl( ) functions. The video stream or image can be obtained from the recognized device, just like from an I/O file.

The UVC device could be listed by shell command devs:

```
-> devs
drv name
0 /null
1 /ttyS0
2 /pcConsole/0
9 /input/event
11 host:
4 /bd0
9 /input/event0
12 /uvc/0
value = 35 = 0x23 = '#'
```

To operate a UVC device:

Procedure

1. Open the UVC device with open( ).
2. Send the request using an ioctl( ) with the appropriate command.
3. Close the device using close( ).

Capture the Video Stream

```
#include <vxWorks.h>
#include <ioLib.h>
#include <usb2Video.h>
```

```
STATUS uvcRecord
    (
    char *  pDevName,       /* UVC device name */
    int     nframes,        /* frame number */
    char *  fileName        /* file name */
    )
    {
    int     fd, vfd;
    int     size, bufSize;
    int     frame;
    struct  timespec ts;
    char *  rcvBuf = NULL;

    if ((fileName == NULL) && (pDevName == NULL))
        return ERROR;

    fd = open(fileName, (O_CREAT|O_RDWR), 0);

    if (fd < 0)
        {
        printf("Open %s failed!\r\n", fileName);
        return ERROR;
        }

    vfd = open(pDevName, O_RDONLY, 0);

    if (vfd < 0)
        {
        printf("Open %s failed!\r\n", pDevName);
        close (fd);
        return ERROR;
        }

    if (ioctl(vfd, USB2_VIDEO_IOCTL_GET_BUFFER_SIZE, (void *)&bufSize) == ERROR)
        {
        printf("Get buffer size failed!\r\n");
        close (fd);
        close (vfd);
        return ERROR;
        }

    rcvBuf = malloc (bufSize);

    if (rcvBuf == NULL)
        {
        printf("Create receive buffer failed!\r\n");
        close (fd);
        close (vfd);
        return ERROR;
        }

    for (frame = 0; frame < nframes; )
        {
        size = read(vfd, rcvBuf, bufSize);

        if (size < 1)
```

```
            {
            printf("Read %s %d failed\r\n", pDevName, frame);
            continue;
            }

        ioctl(vfd, USB2_VIDEO_IOCTL_GET_TS, &ts);
        printf("sz %d @%d.%03d\n", size, ts.tv_sec, ts.tv_nsec/1000000);
write(fd, rcvBuf, size);
frame++;
}

    if (close(vfd) == ERROR)
{
printf("Close %s failed\r\n", pDevName);
return ERROR;
}

    if (close(fd) == ERROR)
{
        printf("Close %s failed\r\n", fileName);
        return ERROR;
        }

    (void)free(rcvBuf);

    return OK;
    }
```

List the Supported Formats for the Video Device

```
STATUS uvcFormatsList
    (
    char * uvcDevName   /* UVC device name */
    )
    {
    VS_FORMAT_USR_DESC * pArg;
    char    string[5] = {0};
    int     vfd, i;

    vfd = open(uvcDevName, O_RDONLY, 0);

    if (vfd < 0)
        {
        printf("Couldn't open %s\r\n", uvcDevName);
        return ERROR;
        }

    pArg = malloc(sizeof(VS_FORMAT_USR_DESC));

    if(pArg == NULL)
        {
        close(vfd);
        return ERROR;
        }

    memset (pArg, 0, sizeof(VS_FORMAT_USR_DESC));
```

```
    printf("\nVideo Interface Support Frame Format List: \r\n");

    for (i = 0; ;i++)
        {
        pArg->formatIndex = i;

        if (ioctl(vfd, USB2_VIDEO_IOCTL_ENUM_FORMATS, pArg) == ERROR)
            break;

        memcpy(string, &(pArg->fourcc), sizeof(pArg->fourcc));
        printf("\t%s  format%d: %s\tcode: 0x%08x subtype: 0x%02x\r\n",
uvcDevName, i, string, pArg->fourcc, pArg->subtype);
        }

    free (pArg);
    close(vfd);

    return OK;
    }
```

List the Supported Frames for the Video Input Device

```
STATUS uvcFramesList
    (
    char *  uvcDevName, /* UVC device name */
    UINT32  fourcc      /* video format */
    )
    {
    VS_FRAME_SIZE_DESC * pArg;
    VS_FRAME_INVL_DESC * pFmInvl;
    int vfd, i, j;

    vfd = open(uvcDevName, O_RDONLY, 0);

    if (vfd < 0)
        {
        printf("Couldn't open %s\r\n", uvcDevName);
        return ERROR;
        }

    pArg = malloc(sizeof(VS_FRAME_SIZE_DESC));

    pFmInvl = malloc(sizeof(VS_FRAME_INVL_DESC));

    if ((pArg == NULL) && (pFmInvl == NULL))
        {
        close(vfd);
        return ERROR;
        }

    memset (pArg, 0, sizeof(VS_FRAME_SIZE_DESC));
    memset (pFmInvl, 0, sizeof(VS_FRAME_INVL_DESC));

    printf("\nVideo Interface Support Frames List: \r\n");
```

```
    for(i = 0; ;i++)
        {
        pArg->fourcc = fourcc;
        pArg->frameIndex = i;

        if (ioctl(vfd, USB2_VIDEO_IOCTL_ENUM_FRAMES_SIZE, pArg) == ERROR)
            break;

        printf("\n\tFrame(%d) Dimension: %4d x %4d \r\n", i+1,
            pArg->width, pArg->height);

        for (j = 0; ;j++)
            {
            pFmInvl->fourcc = pArg->fourcc;
            pFmInvl->width  = pArg->width;
            pFmInvl->height = pArg->height;
            pFmInvl->index  = j;

            if (ioctl(vfd, USB2_VIDEO_IOCTL_ENUM_FRAMES_INVL, pFmInvl) == ERROR)
                break;

            if (pFmInvl->invlType == UVC_FRMIVAL_DISCRETE_TYPE)
                {
                /* Discrete frame intervals */

                printf("\t\t Frame Interval[%d]: %dus Pcs: %d \r\n", j+1,
                           pFmInvl->invlVal, 1000000/pFmInvl->invlVal);
                }
           else
                {
                /* Continuous frame interval */

                printf("\t\t MinFrameInterval: %dus: "
                           " MaxFrameInterval: %dus: "
                           " FrameIntervalStep: %dus \r\n",
                           pFmInvl->contFrameInvl.minInvl,
                           pFmInvl->contFrameInvl.maxInvl,
                           pFmInvl->contFrameInvl.invlStep);
                printf("\t\t Min Pcs: %d: "
                           " Max Pcs: %d: "
                           " Pcs step: %d \r\n",
                           1000000/pFmInvl->contFrameInvl.minInvl,
                           1000000/pFmInvl->contFrameInvl.maxInvl,
                           1000000/pFmInvl->contFrameInvl.invlStep);
                }
            }
        }

    free (pArg);
    close(vfd);

    return OK;
    }
```

Set the Video Input Device Frame Interval

```
STATUS uvcIntervalSet
    (
    char *  pDevName,   /* UVC device name */
    UINT32  interval    /* frame interval */
    )
    {
    int vfd = open (pDevName, O_RDONLY, 0);

    if (vfd < 0)
        {
        printf("Open %s failed!\r\n", pDevName);
        return (ERROR);
        }

    if (ioctl(vfd, USB2_VIDEO_IOCTL_SET_FRAME_INT, (void *)&interval) == ERROR)
        {
        printf("Set frame interval for %s failed!\r\n", pDevName);

        close (vfd);
        return (ERROR);
        }

    printf("%s frame interval = %d us\r\n", pDevName, interval);

    close (vfd);

    return OK;
    }
```

Set the Video Input Device Frame Interval

```
STATUS uvcFormatSet
    (
    char *    pDevName, /* UVC device name */
    UINT32    fourcc,   /* video format */
    UINT16    width,
    UINT16    height
    )
    {
    VS_FORMAT_USR_DATA fmtUsrData;
    STATUS status;

    int vfd = open (pDevName, O_RDONLY, 0);

    if (vfd < 0)
        {
        printf("Open %s failed!\r\n", pDevName);
        return ERROR;
        }

    fmtUsrData.fourcc = fourcc;
    fmtUsrData.width  = width;
    fmtUsrData.height = height;
```

```
    status = ioctl(vfd, USB2_VIDEO_IOCTL_SET_FORMAT,
(VS_FORMAT_USR_DATA *)&fmtUsrData);

    if (status == OK)
        printf("\tformat 0x%08x width = %d height = %d\r\n",
fmtUsrData.fourcc, fmtUsrData.width, fmtUsrData.height);
    else
        printf("Video Input Interface format set failed!\r\n");

    close (vfd);
    return status;
```

Get the Maximum Frame Size for the UVC

```
UINT32 uvcBufSizeGet
    (
    char *  pDevName    /* UVC device name */
    )
    {
    UINT32   size;

    int vfd = open (pDevName, O_RDONLY, 0);

    if (vfd < 0)
        return (-1);

    ioctl(vfd, USB2_VIDEO_IOCTL_GET_BUFFER_SIZE, (void *)&size);

    close (vfd);
    printf("the maximum frame buffer size is %d\r\n", size);

    return size;
    }
```

Selecting File Descriptors

The select( ) function allows a task to pend until one of a set of file descriptors becomes ready. In general, two of its parameters—<pReadFds> and <pWriteFds>—point to file descriptor sets in which each bit corresponds to a particular file descriptor. Bits set in the read file descriptor set (<pReadFds>) cause select( ) to pend until data is available on any of the corresponding file descriptors, while bits set in the write file descriptor set (<pWriteFds>) cause select( ) to pend until any of the corresponding file descriptors become writable.

However, select( ) is not implemented with a separate iosLib method but rather using the FIOSELECT/FIOUNSELECT handlers in the ioctl( ) method. The USB video device can only be mapped as a device file, so <pWriteFds> is not supported on USB video device. In applications, once the USB video file descriptor is selected, the UVC's ioctl( ) handlers received a FIOSELECT command, VS interface is engaged and the process in select( ) is woken up.

The following code example demonstrates this practice.

```
……
/* open USB video device */
vfd = open ("/uvc/0", O_RDONLY|O_NONBLOCK, 0);
```

```
FD_ZERO(&rfdset);
FD_SET(vfd, &rfdset);

/* select video device file descriptor and wait */

if (select ((vfd+1), (FAST fd_set *)(&rfdset), NULL, NULL, NULL) < 0)
    return ERROR;

/* read video frame data from the video device */

size = read(vfd, vbuf, VBUFF_SIZE);
……
```